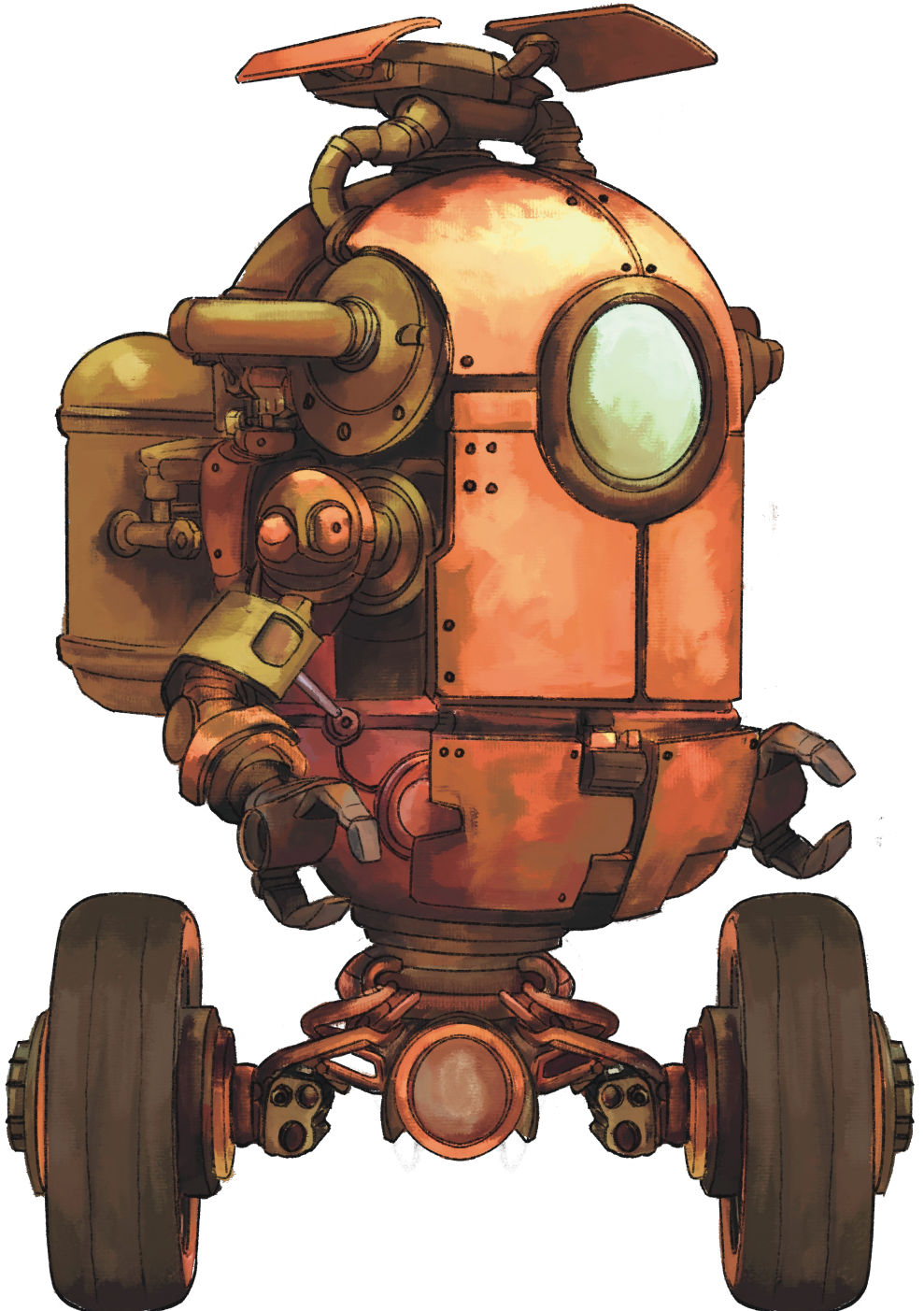
CS 3630, Fall 2026

Lecture 21:

Learning for Robotics



Many slides adapted from Stanford's CS231N by Fei-Fei Li, Justin Johnson, Serena Yeung, as well as Slides by Marc'Aurelio Ranzato (NYU), Dhruv Batra & Devi Parikh (Georgia Tech)

Outline

1. Intra-class variability
2. Supervised Learning
3. Models and Losses
4. Gradient Descent and Minibatches
5. Calculating Gradients
6. Transformers in Robotics

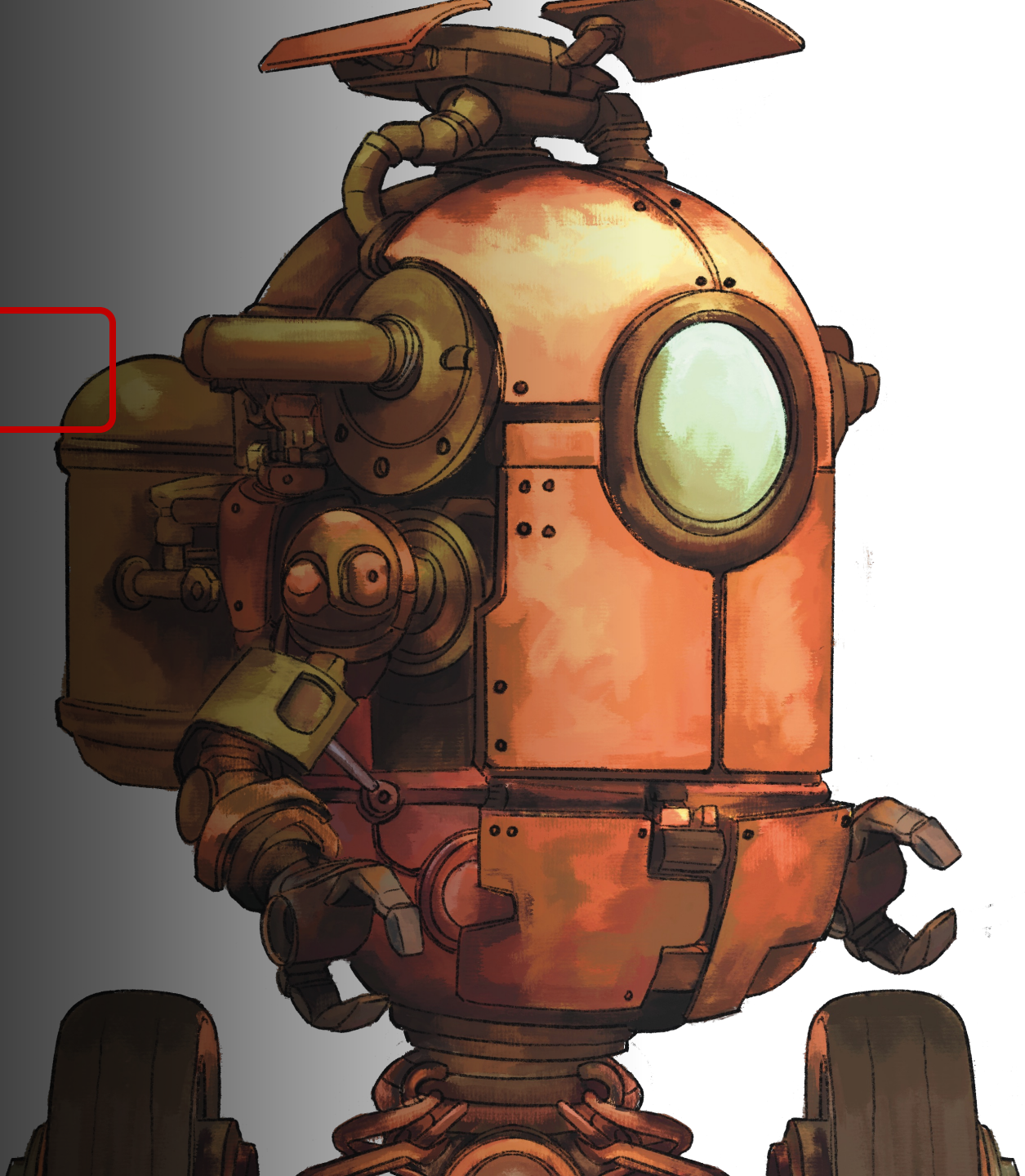


Image Classification: A core task in Computer Vision



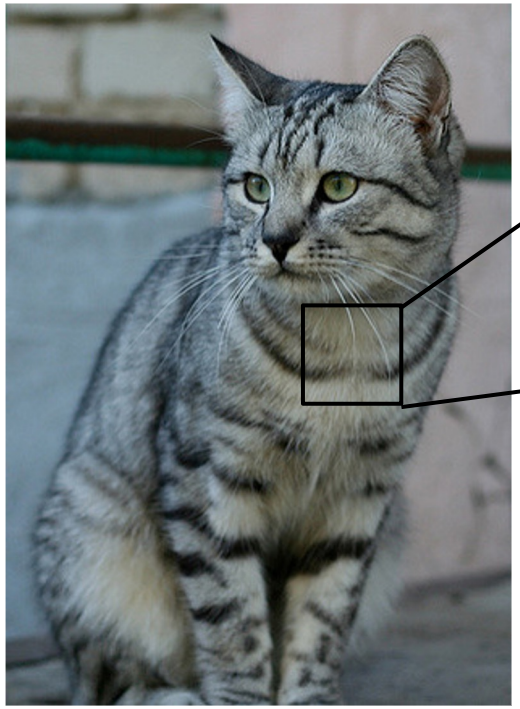
This image by Nikita is licensed under CC-BY 2.0

(assume given set of discrete labels)
{dog, cat, truck, plane, ...}



cat

The Problem: Semantic Gap



This image by Nikita is licensed under CC-BY 2.0

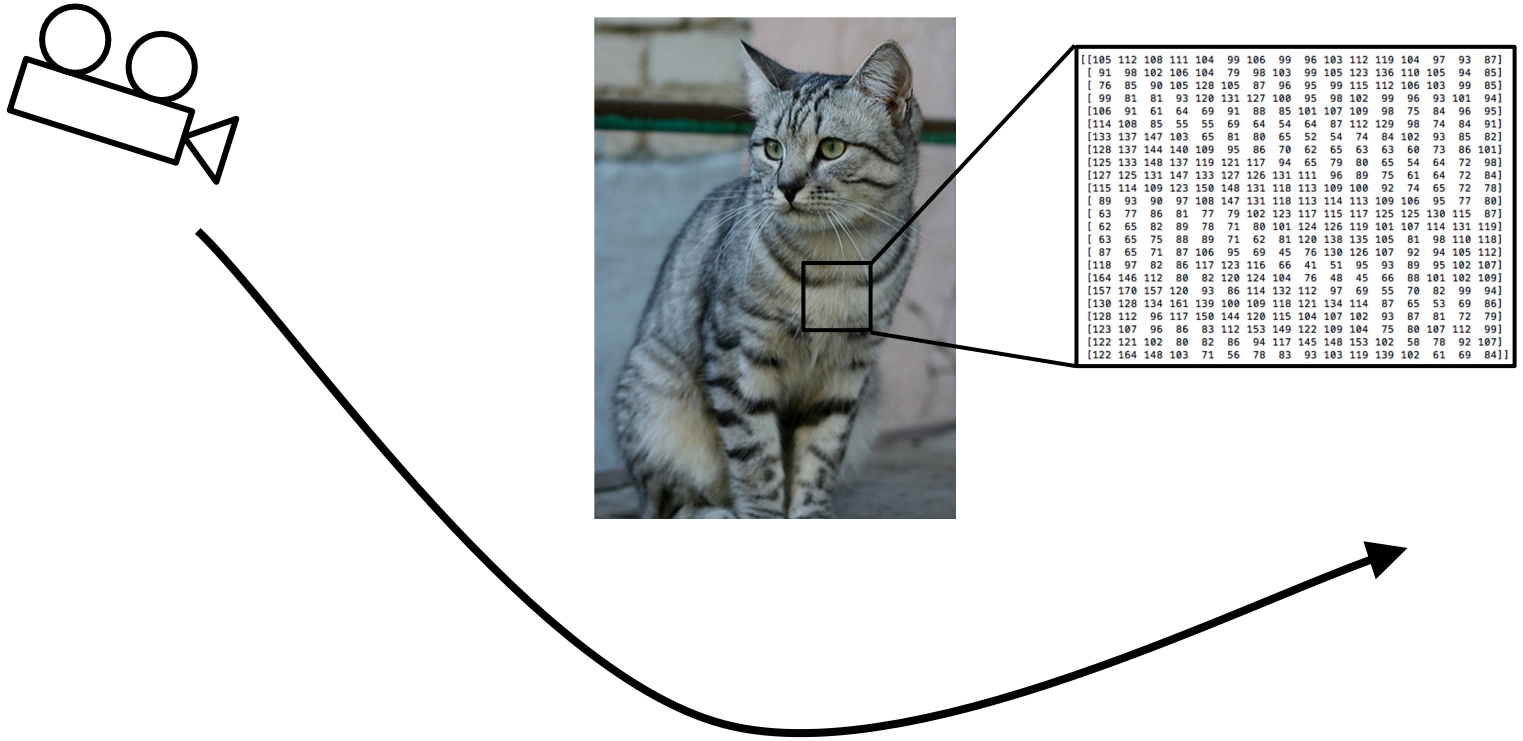
```
[[105 112 108 111 104  99 106  99  96 103 112 119 104  97  93  87]
 [ 91  98 102 106 104  79  98 103  99 105 123 136 110 105  94  85]
 [ 76  85  90 105 128 105  87  96  95  99 115 112 106 103  99  85]
 [ 99  81  81  93 120 131 127 100  95  98 102  99  96  93 101  94]
 [106  91  61  64  69  91  88  85 101 107 109  98  75  84  96  95]
 [114 108  85  55  55  69  64  54  64  87 112 129  98  74  84  91]
 [133 137 147 103  65  81  80  65  52  54  74  84 102  93  85  82]
 [128 137 144 140 109  95  86  70  62  65  63  63  60  73  86 101]
 [125 133 148 137 119 121 117  94  65  79  80  65  54  64  72  98]
 [127 125 131 147 133 127 126 131 111  96  89  75  61  64  72  84]
 [115 114 109 123 150 148 131 118 113 109 100  92  74  65  72  78]
 [ 89  93  90  97 108 147 131 118 113 114 113 109 106  95  77  80]
 [ 63  77  86  81  77  79 102 123 117 115 117 125 125 130 115  87]
 [ 62  65  82  89  78  71  80 101 124 126 119 101 107 114 131 119]
 [ 63  65  75  88  89  71  62  81 120 138 135 105  81  98 110 118]
 [ 87  65  71  87 106  95  69  45  76 130 126 107  92  94 105 112]
 [118  97  82  86 117 123 116  66  41  51  95  93  89  95 102 107]
 [164 146 112  80  82 120 124 104  76  48  45  66  88 101 102 109]
 [157 170 157 120  93  86 114 132 112  97  69  55  70  82  99  94]
 [130 128 134 161 139 100 109 118 121 134 114  87  65  53  69  86]
 [128 112  96 117 150 144 120 115 104 107 102  93  87  81  72  79]
 [123 107  96  86  83 112 153 149 122 109 104  75  80 107 112  99]
 [122 121 102  80  82  86  94 117 145 148 153 102  58  78  92 107]
 [122 164 148 103  71  56  78  83  93 103 119 139 102  61  69  84]]
```

What the computer sees

An image is just a big grid of numbers between [0, 255]:

e.g. 800 x 600 x 3
(3 channels RGB)

Challenges: Viewpoint variation



All pixels change when the camera moves!

This image by Nikita is licensed under CC-BY 2.0

Challenges: Illumination



This image is CC0 1.0 public domain



This image is CC0 1.0 public domain



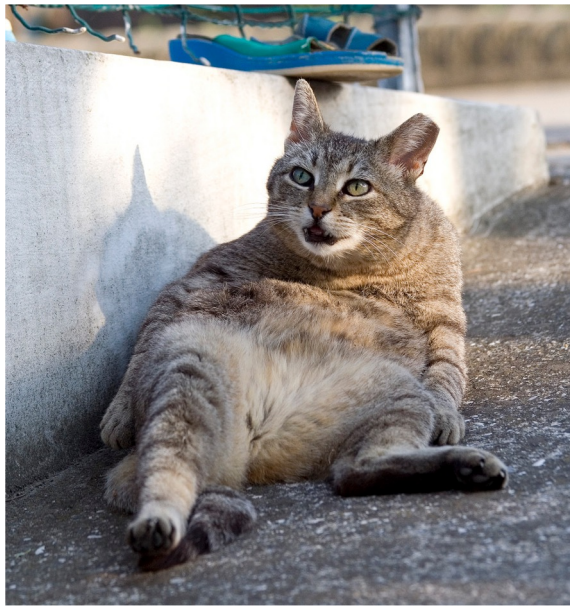
This image is CC0 1.0 public domain



This image is CC0 1.0 public domain

Slide Credit: Fei-Fei Li, Justin Johnson, Serena Yeung, CS 231n

Challenges: Deformation



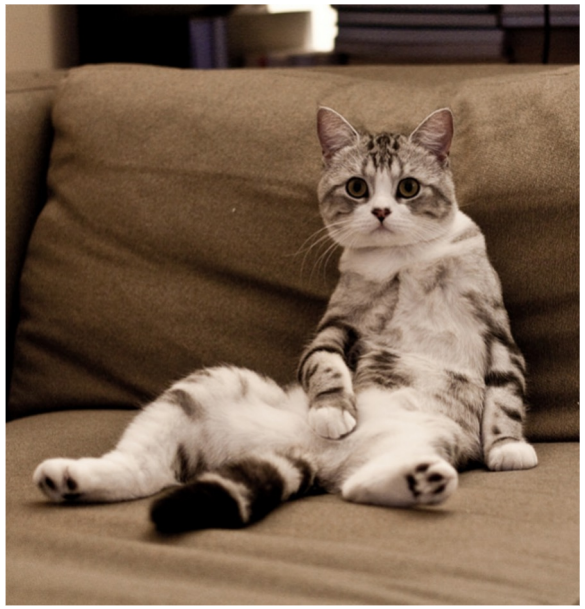
This image by Umberto Salvagnin is licensed under CC-BY 2.0



This image by Umberto Salvagnin is licensed under CC-BY 2.0



This image by sare bear is licensed under CC-BY 2.0



This image by Tom Thai is licensed under CC-BY 2.0

Slide Credit: Fei-Fei Li, Justin Johnson, Serena Yeung, CS 231n

Challenges: Occlusion



This image is CC0 1.0 public domain



This image is CC0 1.0 public domain



This image by jonsson is licensed under CC-BY 2.0

Slide Credit: Fei-Fei Li, Justin Johnson, Serena Yeung, CS 231n

Challenges: Background Clutter



This image is CC0 1.0 public domain



This image is CC0 1.0 public domain

Slide Credit: Fei-Fei Li, Justin Johnson, Serena Yeung, CS 231n

Challenges: Intraclass variation

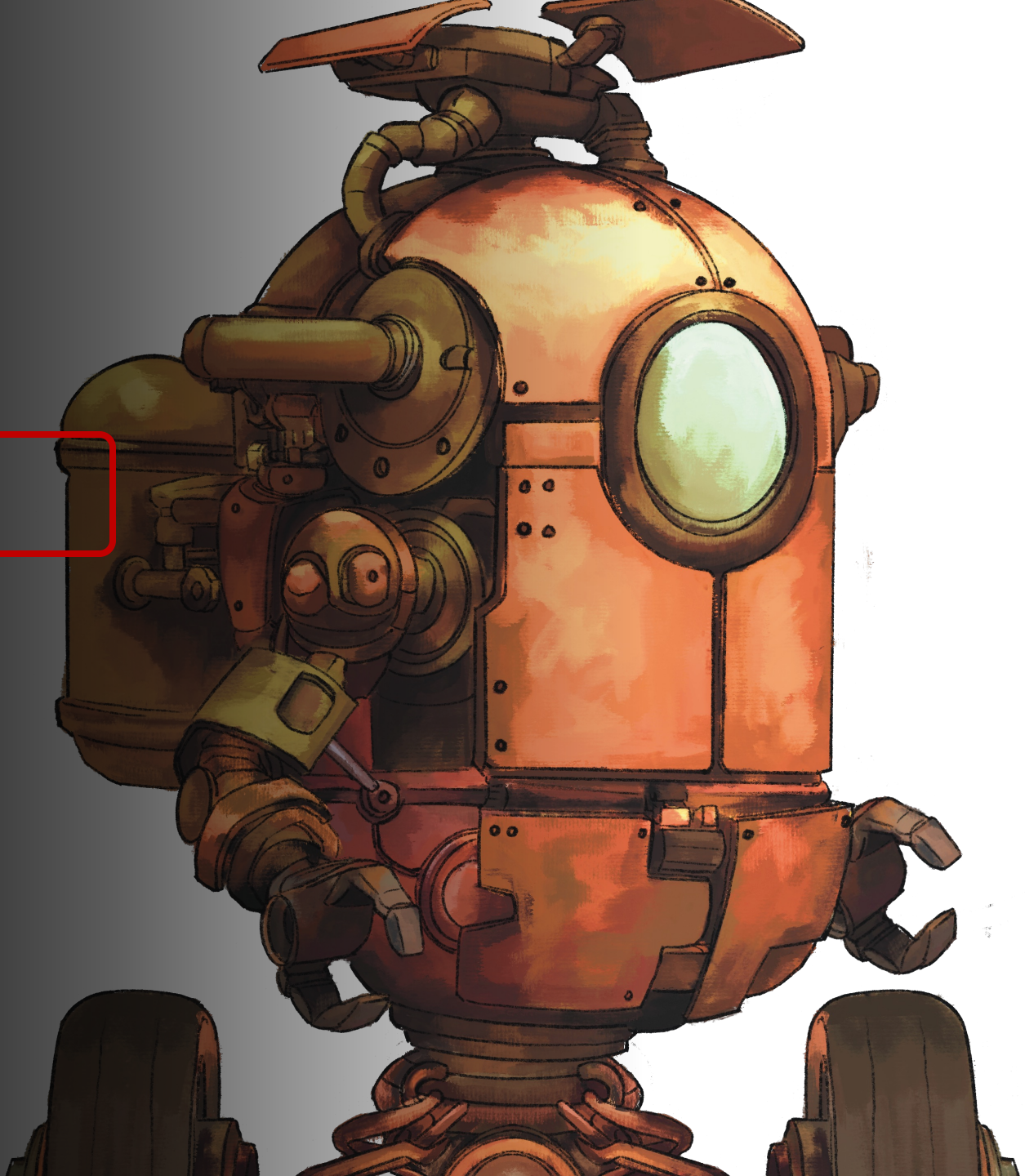


This image is CC0 1.0 public domain

Slide Credit: Fei-Fei Li, Justin Johnson, Serena Yeung, CS 231n

Outline

1. Intra-class variability
2. Supervised Learning
3. Models and Losses
4. Gradient Descent and Minibatches
5. Calculating Gradients
6. Transformers in Robotics



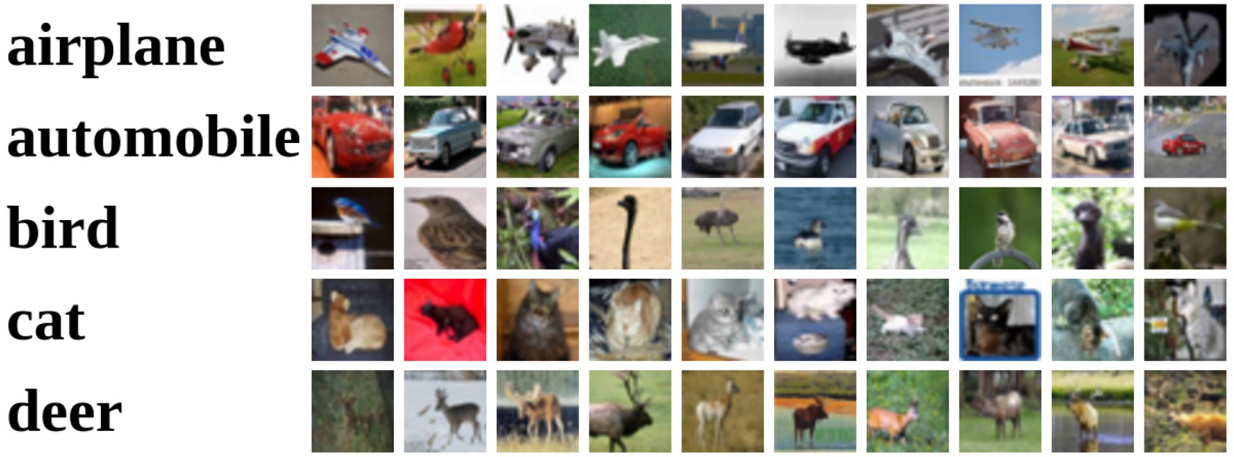
ML: A Data-Driven Approach

1. Collect a dataset of images x and labels y
2. Use Machine Learning to train a classifier
3. Evaluate the classifier on new images

```
def train(images, labels):
  # Machine learning!
  return model
```

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```

Example training set



Slide Credit: Fei-Fei Li, Justin Johnson, Serena Yeung, CS 231n

Supervised Learning Setup

Each training example pairs an input x with a target y.

$$\text{input } x \;\rightarrow\; \text{model } f(x;\theta) \;\rightarrow\; \text{prediction } \hat{y}$$

Classification

Camera image → floor type

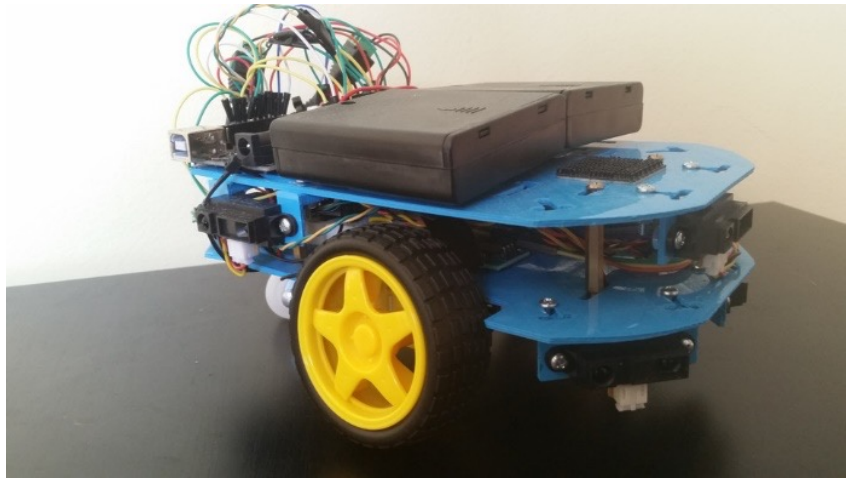
Target: tile or carpet

Regression

Wheel speeds → forward speed

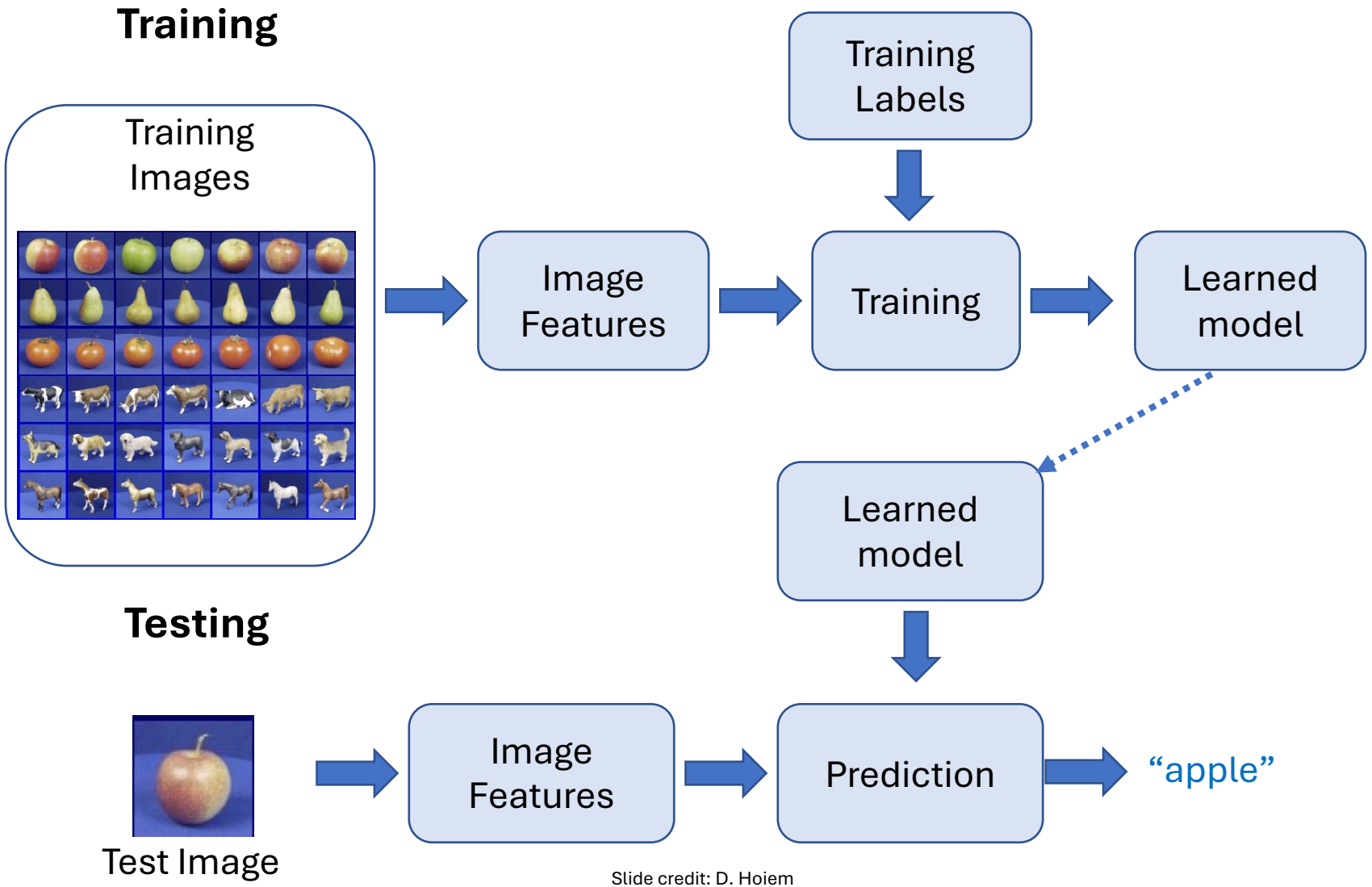
Target: measured speed in m/s

Training adjusts θ to reduce prediction error.



Differential-drive robot • CS 3630

Steps



Slide credit: D. Hoiem

Training, Validation, and Test Data

Use separate datasets to fit, tune, and evaluate the model.

Training

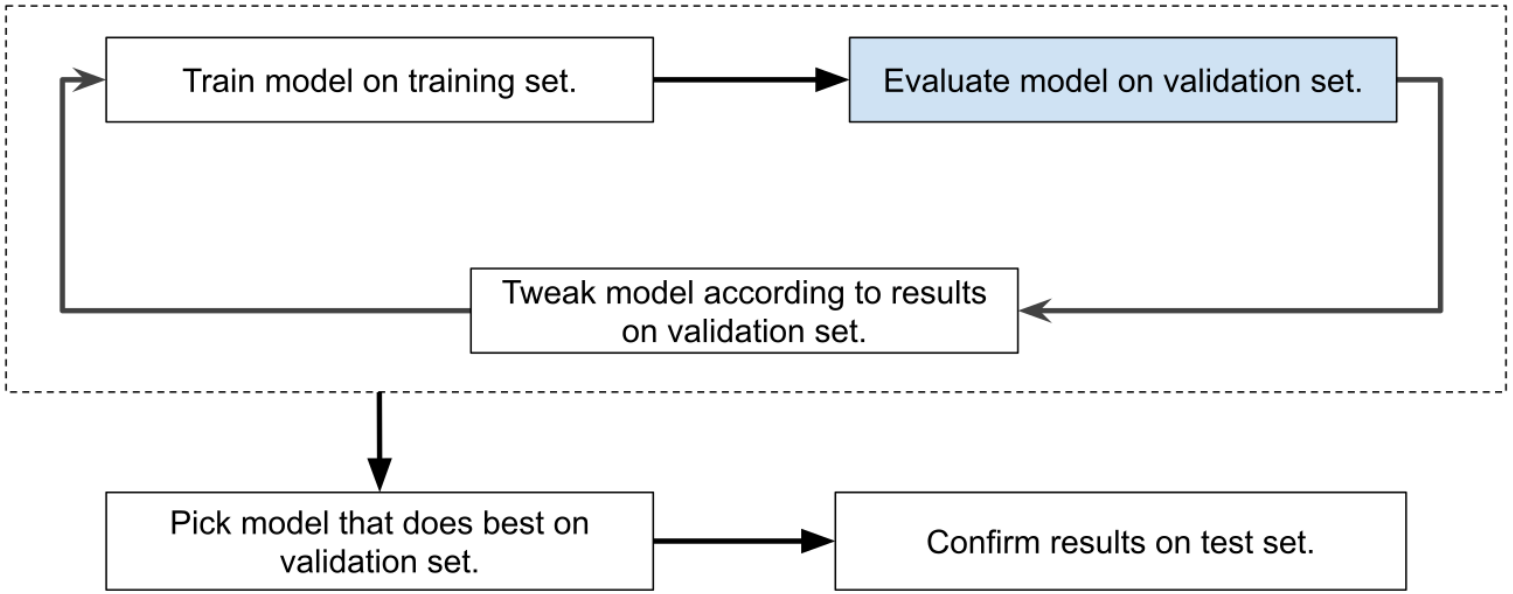
Fit the model's parameters.

Validation

Choose the learning rate, model size, or stopping point.

Test

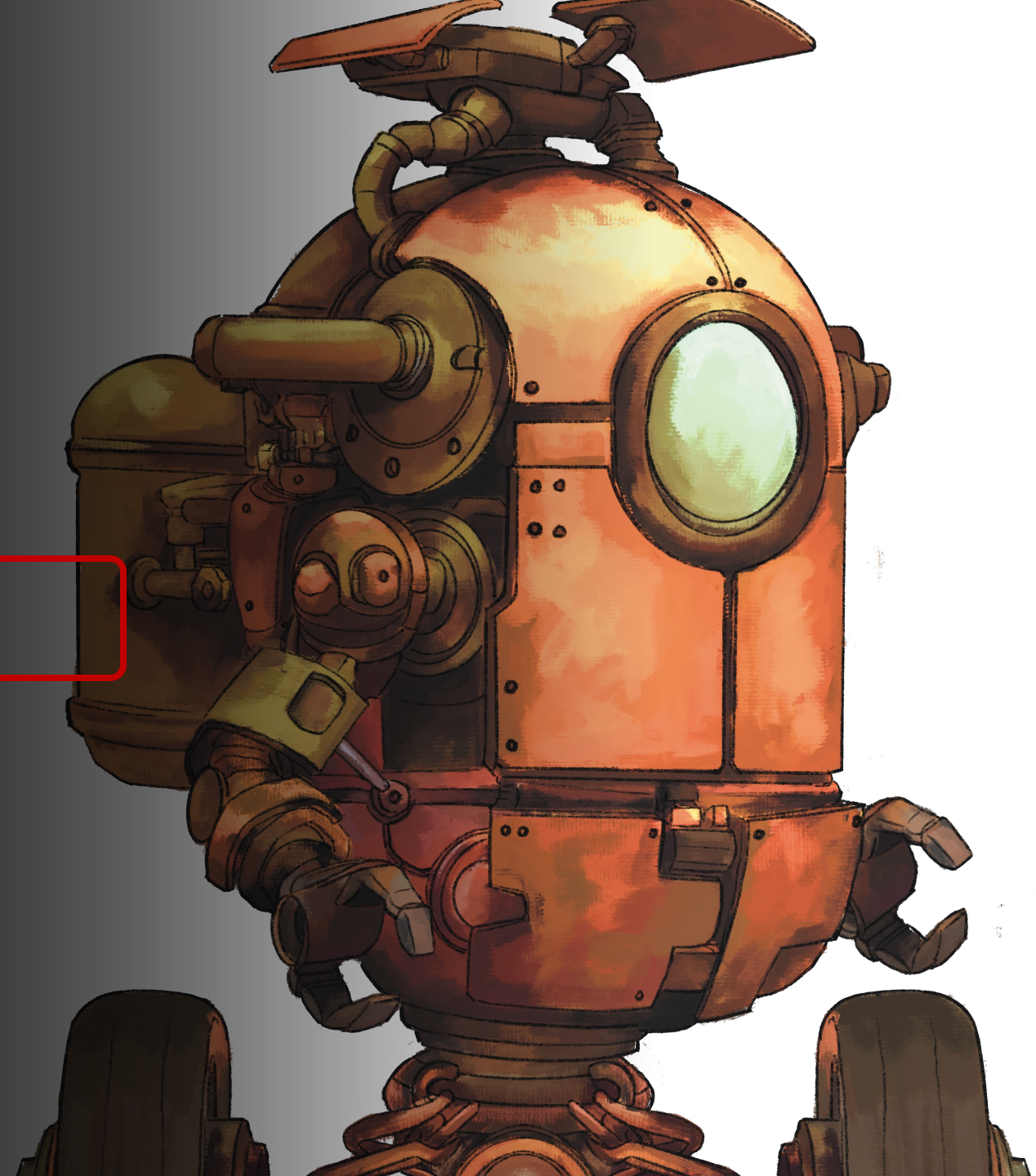
Measure how well the final model handles unseen data.



For robot logs, hold out whole runs to keep nearby frames together.

Outline

1. Intra-class variability
2. Supervised Learning
3. Models and Losses
4. Gradient Descent and Minibatches
5. Calculating Gradients
6. Transformers in Robotics

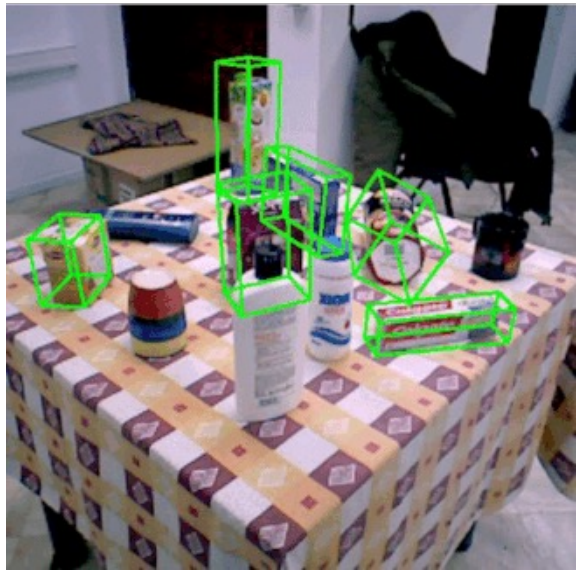


Two different learning problems are classification and regression

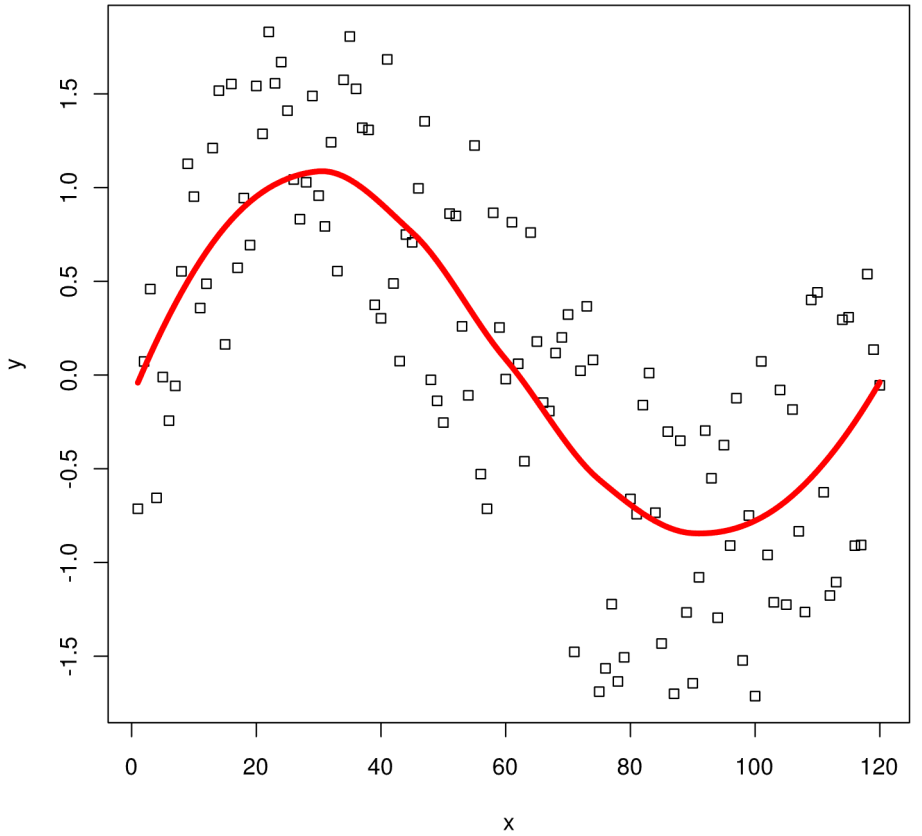
- **Regression**: continuous labels y
- Sum-square-difference loss:

$$L_{\mathrm{SSD}}(W;D) \doteq \sum_{(x,y)\in D} |f(x;W)-y|^2$$

- Example: direct 3D pose regression:



https://paperswithcode.com/task/6d-pose-estimation



By Kierano - Own work, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=7034448

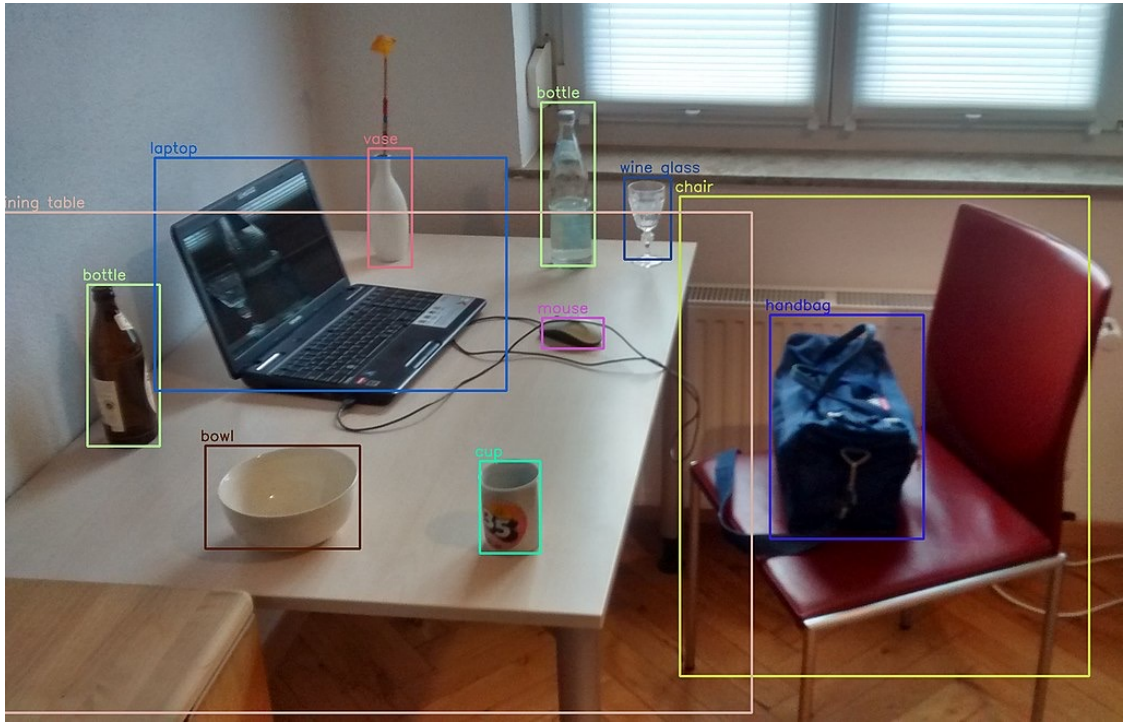
Two different learning problems are classification and regression

- **Classification**: discrete labels y
- Cross-entropy loss:

$$\mathcal{L}_{\mathrm{CE}}(\theta; D) \doteq \sum_{c} \sum_{(x, y=c) \in D} \log \frac{1}{p_c(x; \theta)}$$

- Average surprise!

- Example: object detection:



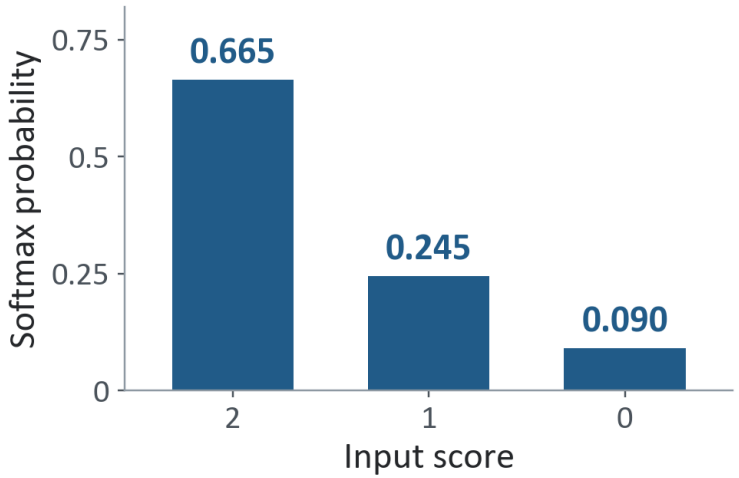
(MTheiler), CC BY-SA 4.0 <https://creativecommons.org/licenses/by-sa/4.0>, via Wikimedia Commons

How to get probabilities? Softmax converts a set of C class “scores” into “probabilities”

- If we need to classify inputs into C different classes, we put C units in the last layer to produce C *one-vs.-others* scores $f_1, f_2, \ldots, f_C$
- Apply *softmax* function to convert these scores to probabilities:

$$\text{softmax}(f_1, \ldots, f_c) = \left(\frac{\exp(f_1)}{\sum_j \exp(f_j)}, \ldots, \frac{\exp(fc)}{\sum_j \exp(f_j)} \right)$$

If one of the inputs is much larger than the others, then the corresponding softmax value will be close to 1 and others will be close to 0



Losses for Regression and Classification

Regression: mean squared error

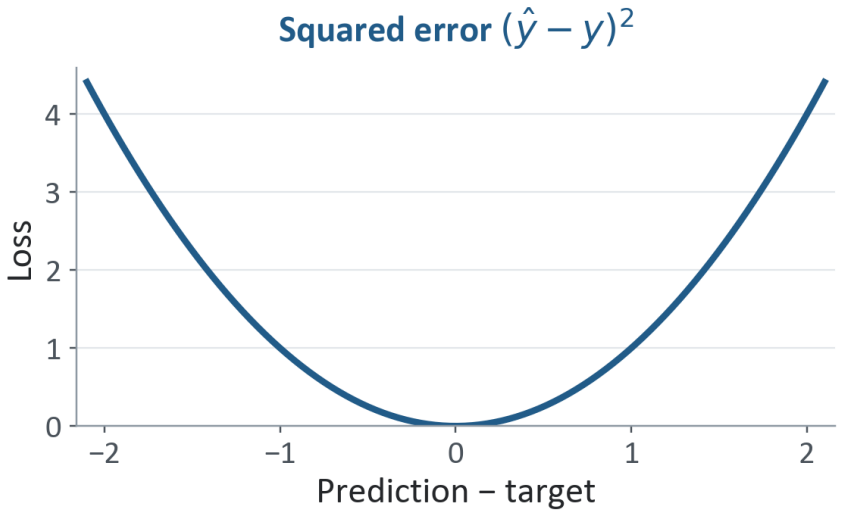
$$\mathrm{L} = (\hat{\mathrm{y}} - \mathrm{y})^2$$

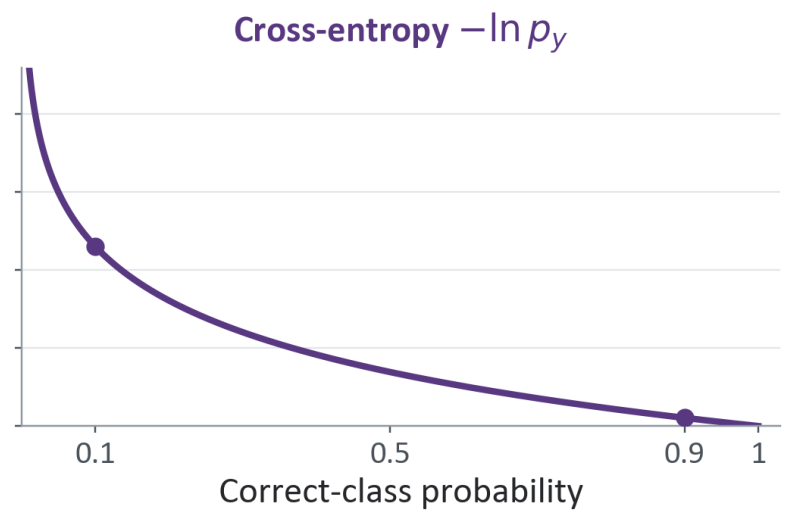
Square each residual and average across outputs and examples.

Classification: cross-entropy

$$L = -\log p_y$$

Use the probability p(y) assigned to the correct class.

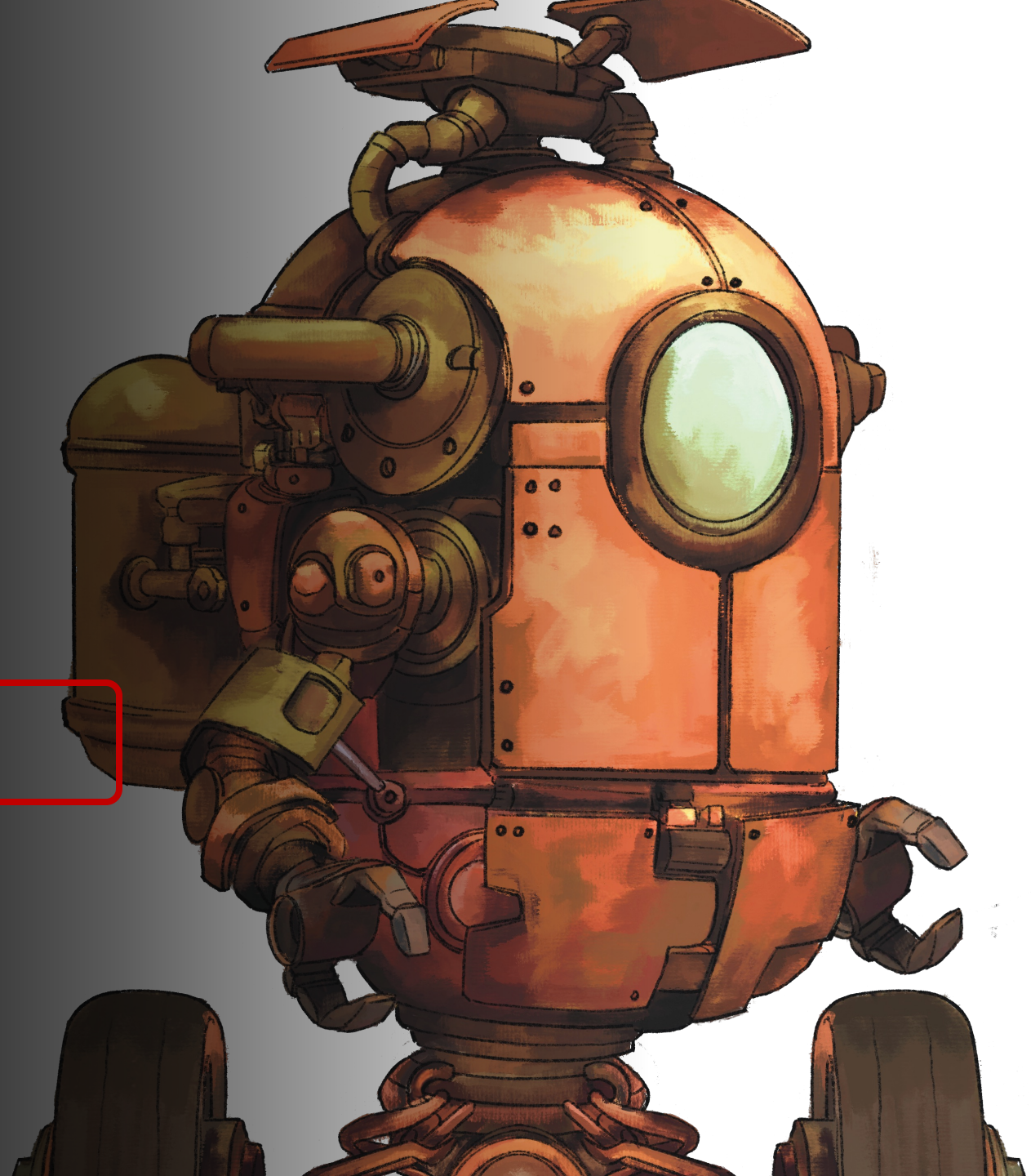




Correct-class probability: 0.9 gives loss 0.11; 0.1 gives loss 2.30.

Outline

1. Intra-class variability
2. Supervised Learning
3. Models and Losses
4. Gradient Descent and Minibatches
5. Calculating Gradients
6. Transformers in Robotics



How to minimize the loss by changing the weights?

Strategy: Follow the slope of the loss function



Slide Credit: Fei-Fei Li, Justin Johnson, Serena Yeung, CS 231n

Strategy: Follow the slope

In 1-dimension, the derivative of a function:

$$\frac{df(x)}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

In multiple dimensions, the **gradient** is the vector of (partial derivatives) along each dimension

The slope in any direction is the **dot product** of the direction with the gradient

The direction of steepest descent is the **negative gradient**

Gradient Descent

```
# Vanilla Gradient Descent

while True:
  weights_grad = evaluate_gradient(loss_fun, data, weights)
  weights += - step_size * weights_grad # perform parameter update
```

Slide Credit: Fei-Fei Li, Justin Johnson, Serena Yeung, CS 231n

Stochastic Gradient Descent (SGD)

$$L(W) = \frac{1}{N}\sum_{i=1}^{N} L_i(x_i, y_i, W) + \lambda R(W)$$

$$\nabla_W L(W) = \frac{1}{N}\sum_{i=1}^{N} \nabla_W L_i(x_i, y_i, W) + \lambda \nabla_W R(W)$$

Full sum expensive when N is large!

Approximate sum using a **minibatch** of examples
32 / 64 / 128 common

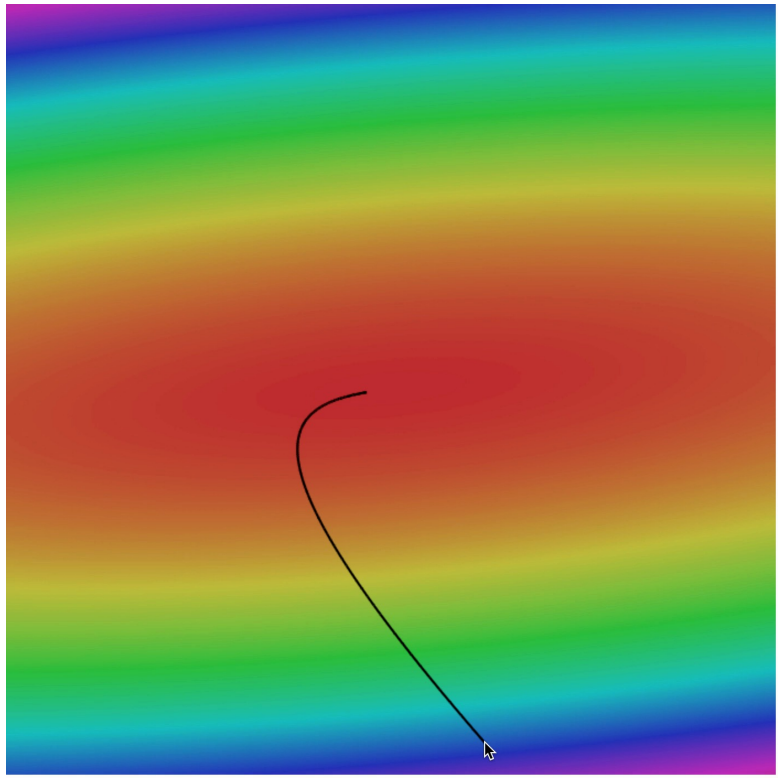
```
# Vanilla Minibatch Gradient Descent

while True:
  data_batch = sample_training_data(data, 256) # sample 256 examples
  weights_grad = evaluate_gradient(loss_fun, data_batch, weights)
  weights += - step_size * weights_grad # perform parameter update
```

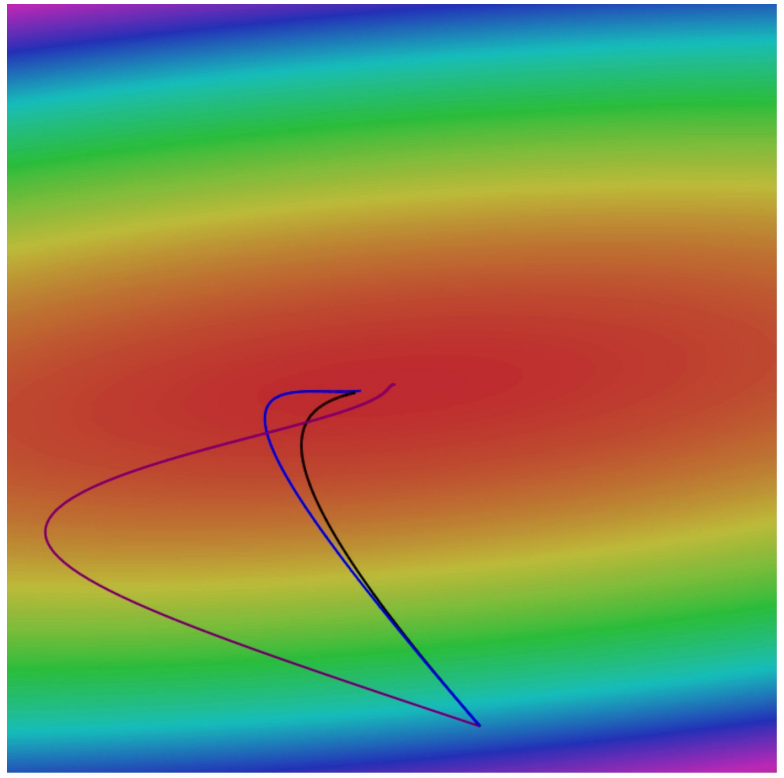
Slide Credit: Fei-Fei Li, Justin Johnson, Serena Yeung, CS 231n

Optimizer Trajectories

Stochastic gradient descent



SGD and Adam

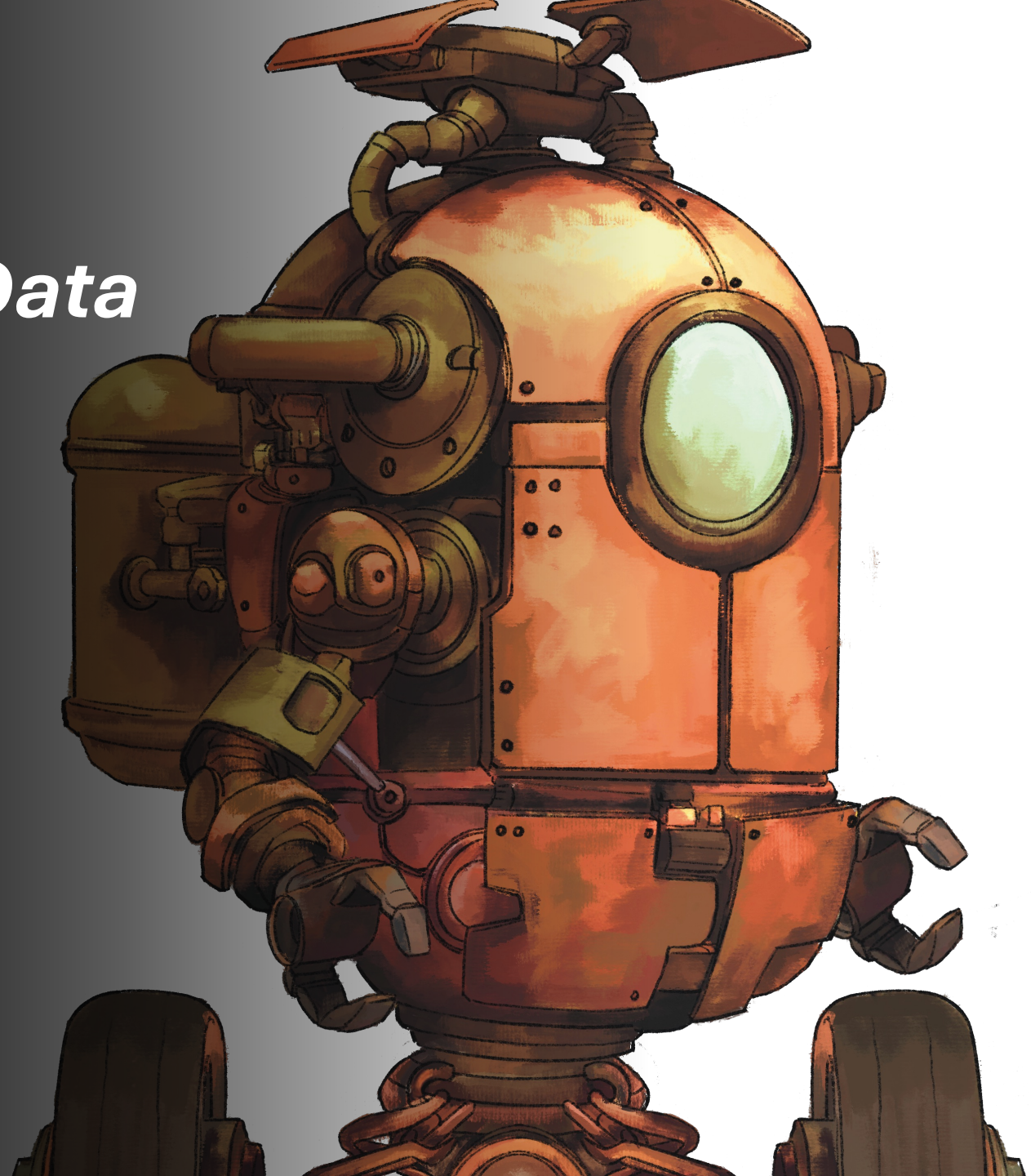


Slide Credit: Fei-Fei Li, Justin Johnson, Serena Yeung, CS 231n

Fitting a Model from Data

A worked example with LineGrid

Robotics Book §5.6



A Learnable 1D Grid

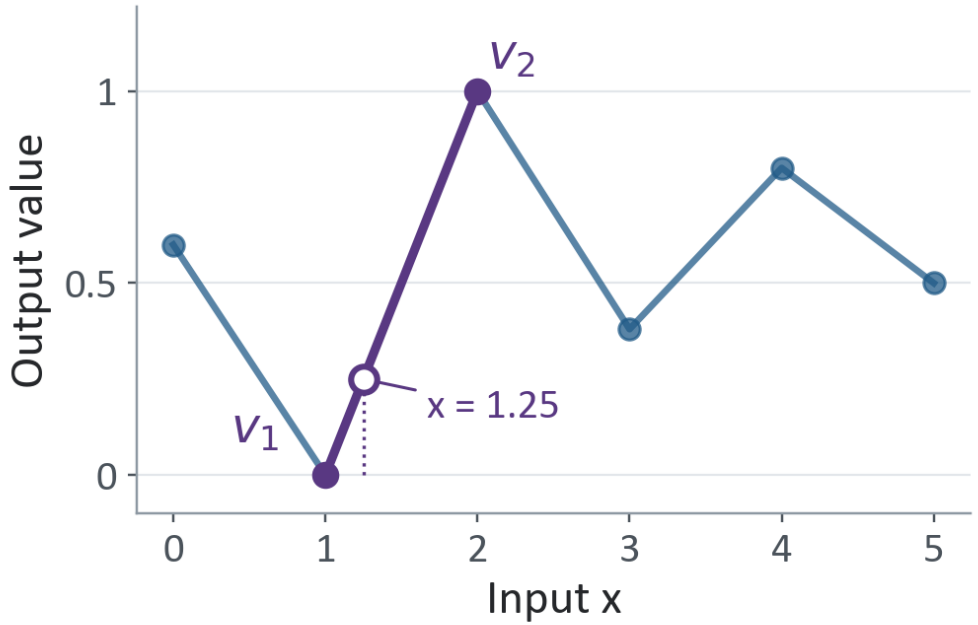
Split the input interval into n cells. Learn a d-vector v at each boundary.

$$0 \quad \cdots \quad i \quad \leq \quad x \quad < \quad i+1 \quad \cdots \quad n$$

$$i = \mathrm{floor}(x), \quad \alpha = x - i, \quad 0 \leq x < n$$

$$f(x;\, \theta) \;=\; (1 - \alpha)\, v_i \;+\; \alpha\, v_{i+1}$$

Only the boundary vectors are learned.



Quick check: with n = 5 and d = 2, how many scalars do we learn?

Learning Sine and Cosine from Noise

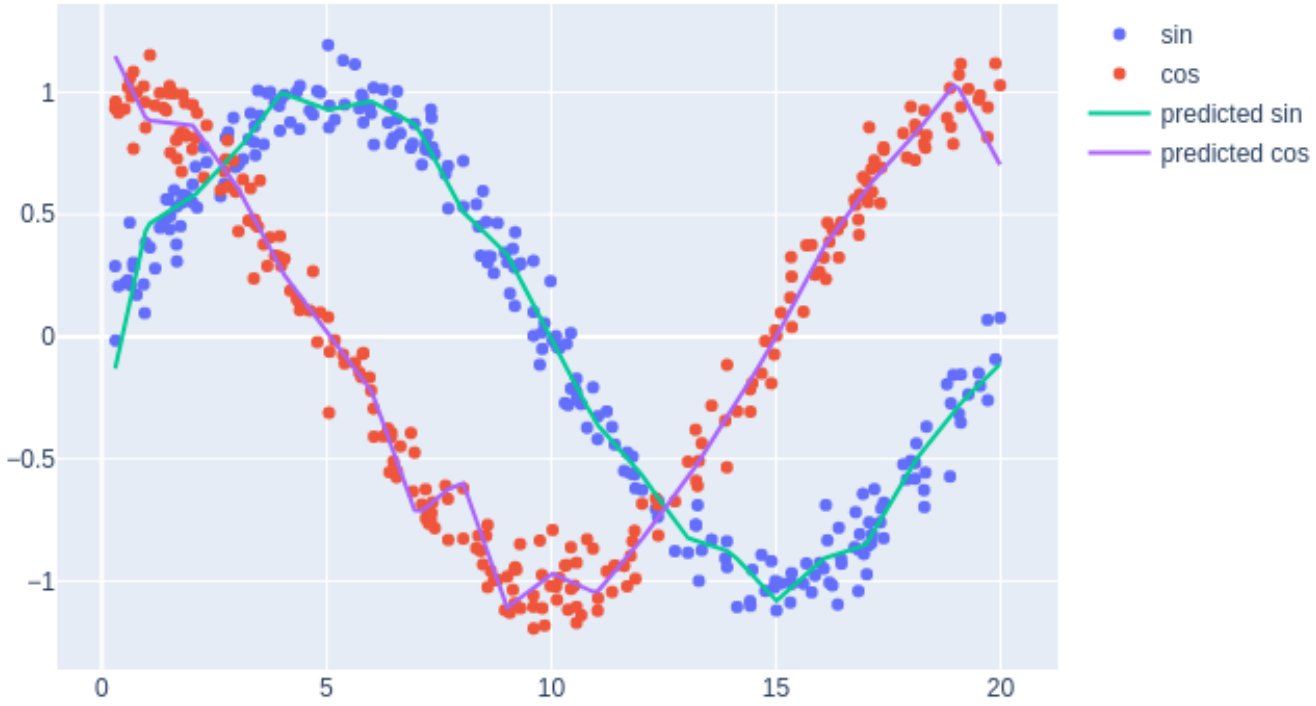
Training data

250 inputs, each with a noisy sine and cosine target.

LineGrid

20 cells, 2 outputs

21 × 2 = 42 parameters

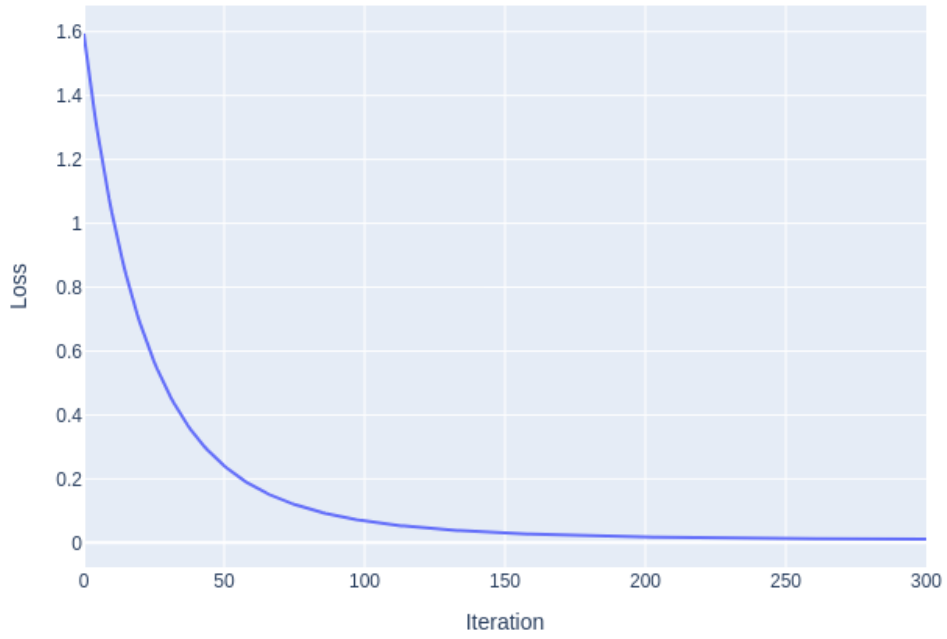


The model learns a piecewise linear approximation to each curve.

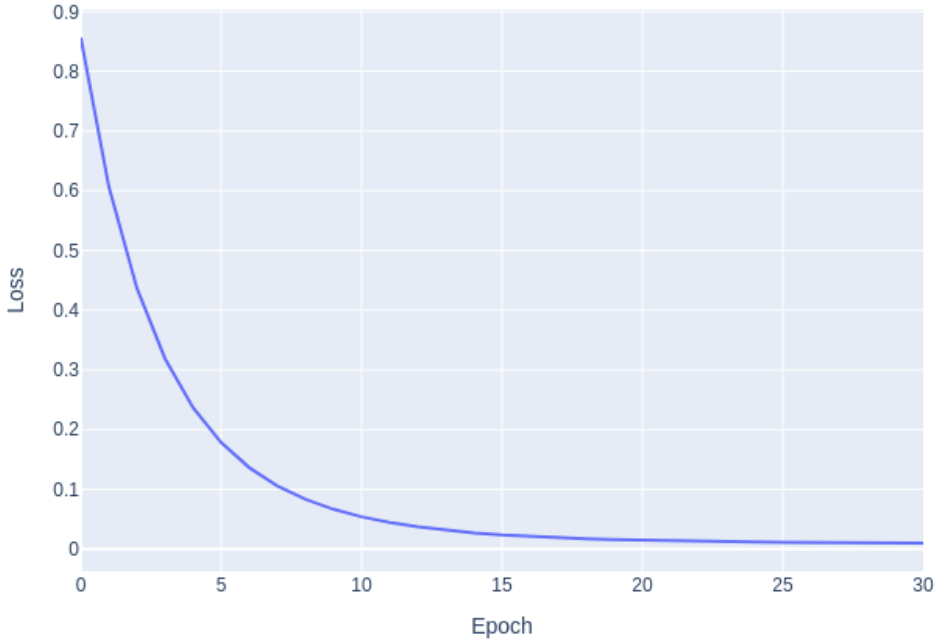
Minibatch Stochastic Gradient Descent

With 250 examples and batches of 25, one epoch contains 10 updates.

Full batch: 1 update per pass



Minibatch: 10 updates per pass



A minibatch gives a cheaper, noisier estimate of the full gradient.

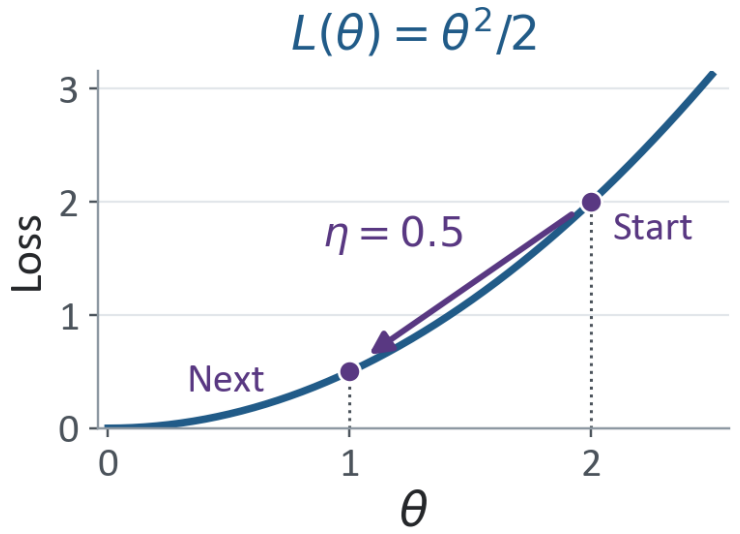
The two runs start from different weights and use different axis scales.

One Gradient Descent Update

Full-batch gradient descent uses all training examples for each update.

```
optimizer.zero_grad()                    Clear old gradients
y_hat = model(x_train)                   Predict
loss = loss_fn(y_hat, y_train)           Measure error
loss.backward()                          Differentiate
optimizer.step()                         Update parameters
```

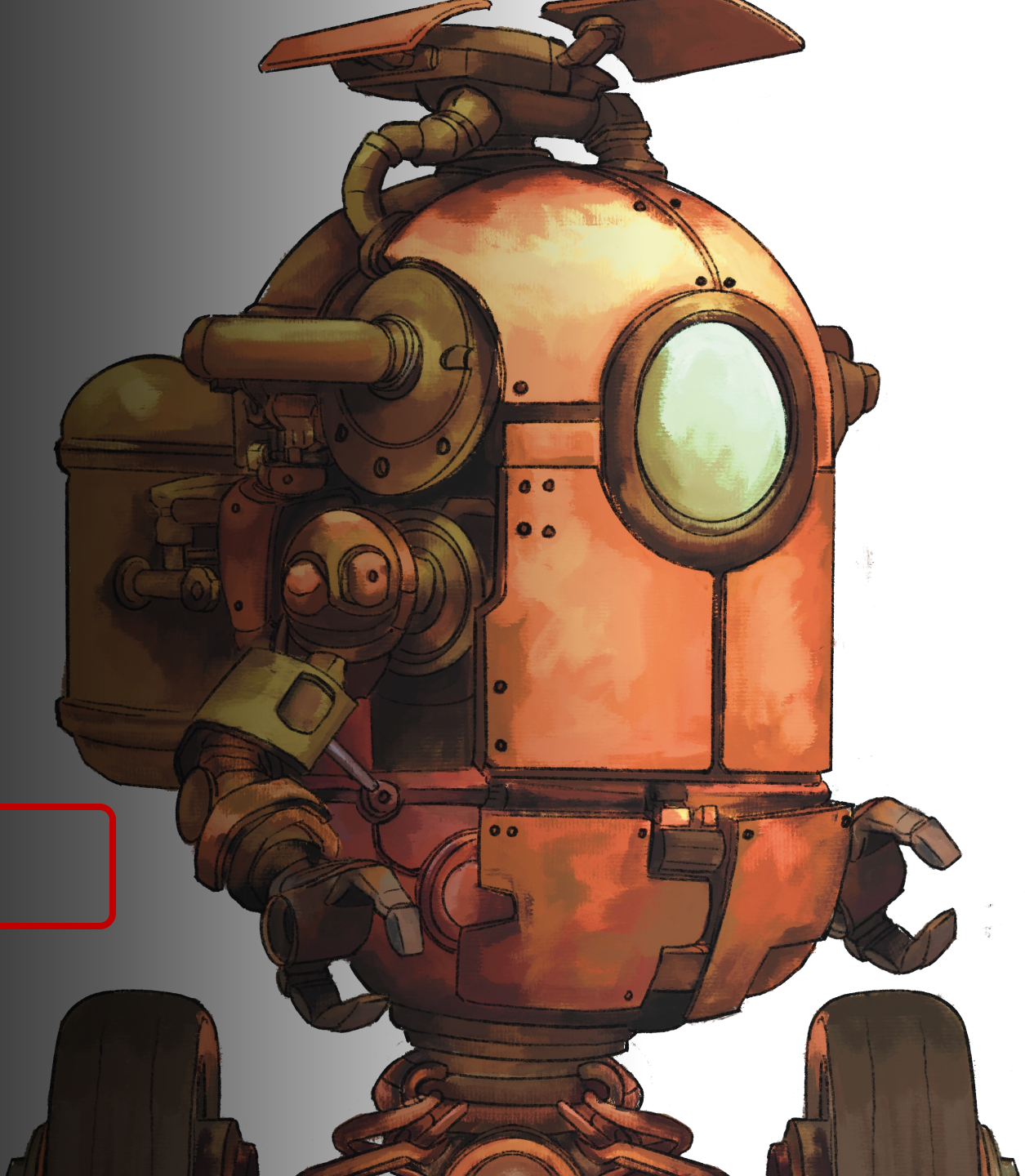
$$\theta \leftarrow \theta - \eta \nabla L(\theta)$$



The learning rate η sets the step size.

Outline

1. Intra-class variability
2. Supervised Learning
3. Models and Losses
4. Gradient Descent and Minibatches
5. Calculating Gradients
6. Transformers in Robotics



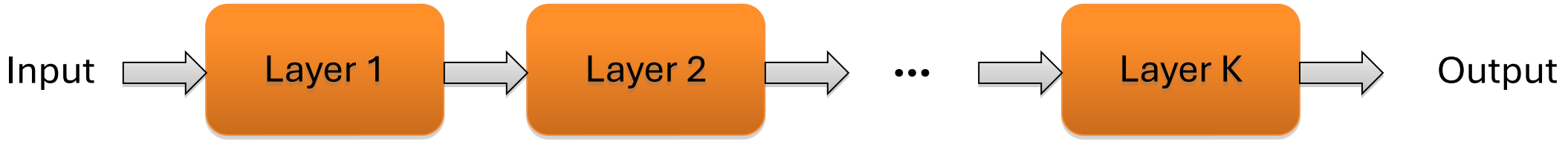
How do we *really* compute gradients?

- Analytic or “Manual” Differentiation
- Symbolic Differentiation
- Numerical Differentiation
- Automatic Differentiation!
  - Forward mode AD
  - Reverse mode AD
    - aka “backpropagation”
  - Implemented in specialized frameworks:
    - PyTorch (Facebook)
    - TensorFlow (Google) frameworks
  - Main computation, mainly done on GPU (or TPU)

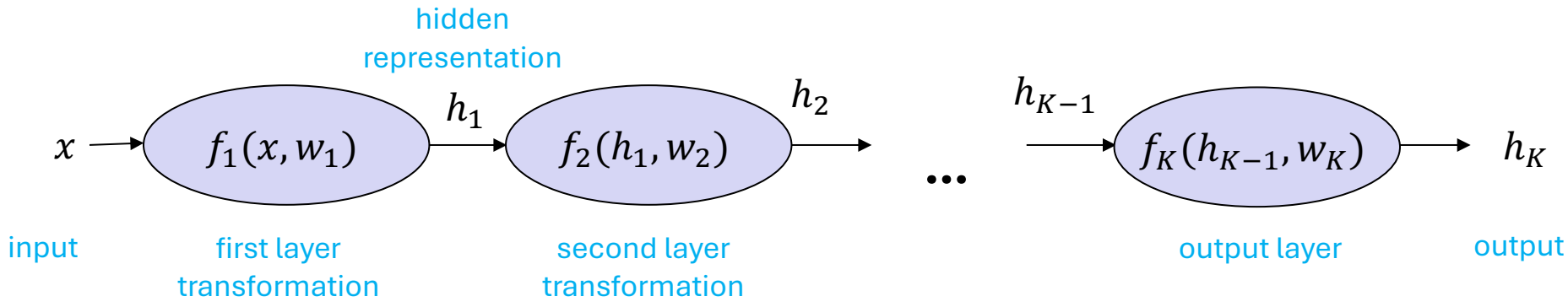


Multilayer Neural Networks

- The function computed by the network is a composition of the functions computed by individual layers (e.g., linear layers and nonlinearities):

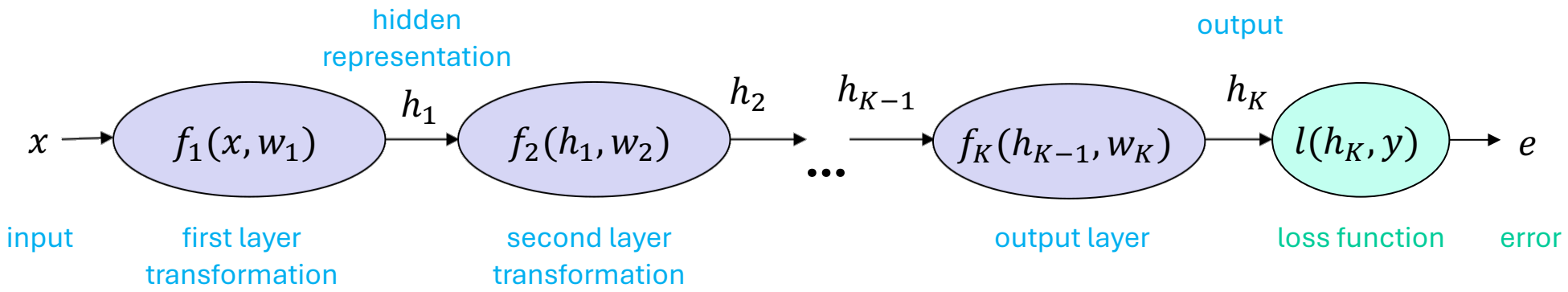


- More precisely:



Adapted from S. Lazebnik, CS 444 (Spring 2025)

Training a Multilayer Network

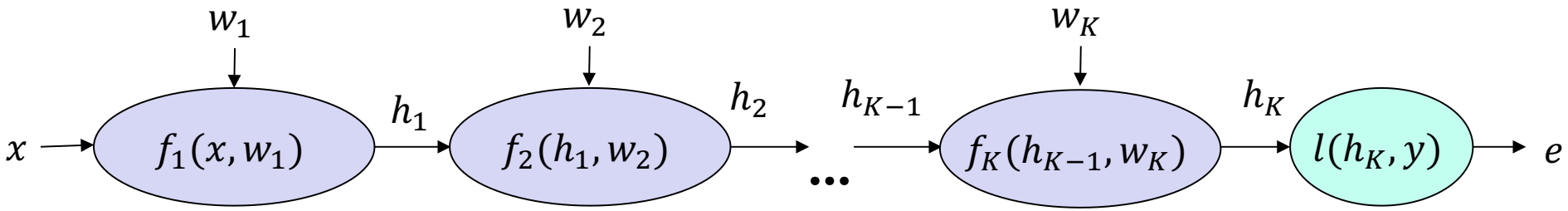


- What is the SGD update for the parameters w_k of the kth layer?

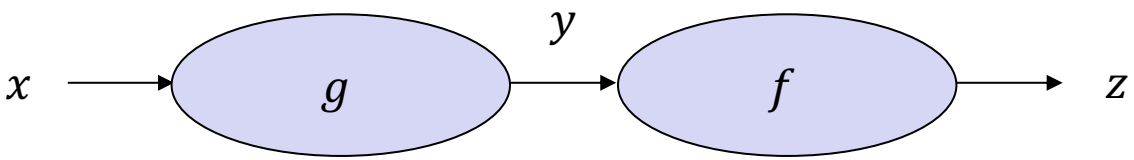
$$w_k \leftarrow w_k - \eta \frac{\partial e}{\partial w_k}$$

- To train the network, we need to find the gradient of the error w.r.t. the parameters of each layer, $\frac{\partial e}{\partial w_k}$

Computation Graph



The Chain Rule



$$\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx}$$

In calculus, the **chain rule** is a formula that expresses the derivative of the composition of two differentiable functions f and g in terms of the derivatives of f and g. More precisely, if $h = f \circ g$ is the function such that $h(x) = f(g(x))$ for every x, then the chain rule is, in Lagrange's notation,

$$h'(x) = f'(g(x))g'(x).$$

or, equivalently,

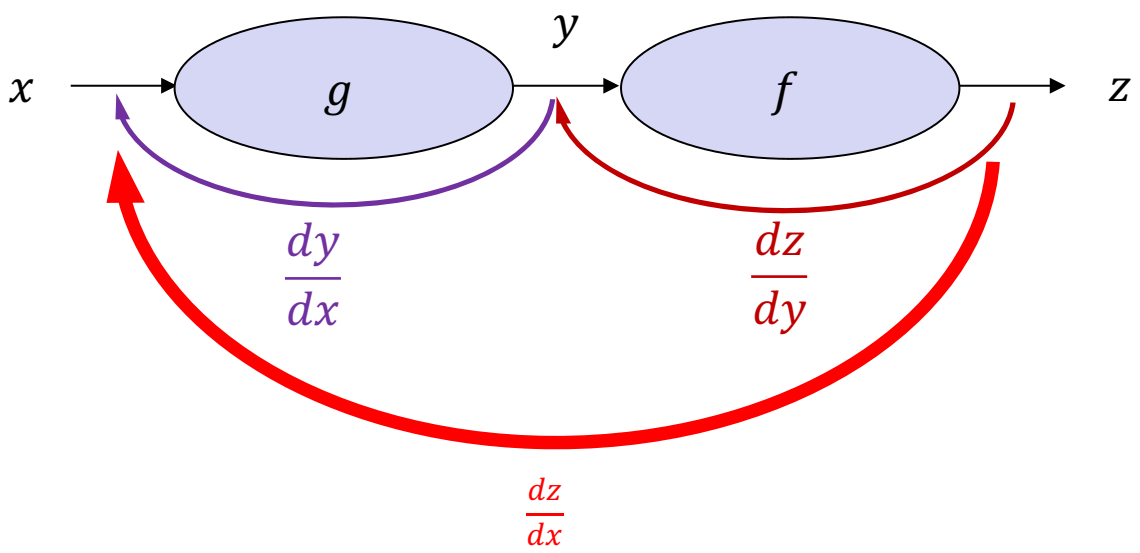
$$h' = (f \circ g)' = (f' \circ g) \cdot g'.$$

The chain rule may also be expressed in Leibniz's notation. If a variable z depends on the variable y, which itself depends on the variable x (that is, y and z are dependent variables), then z depends on x as well, via the intermediate variable y. In this case, the chain rule is expressed as

$$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$$

https://en.wikipedia.org/wiki/Chain_rule

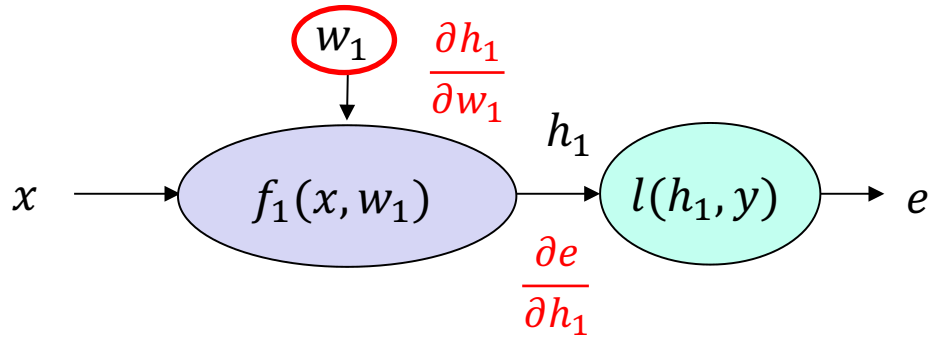
The Chain Rule in a Computation Graph



$$\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx}$$

Applying the Chain Rule

Let's start with $k = 1$



$e = l(f_1(x, w_1), y)$

Example: $e = (y - w_1^T x)^2$

$h_1 = f_1(x, w_1) = w_1^T x$

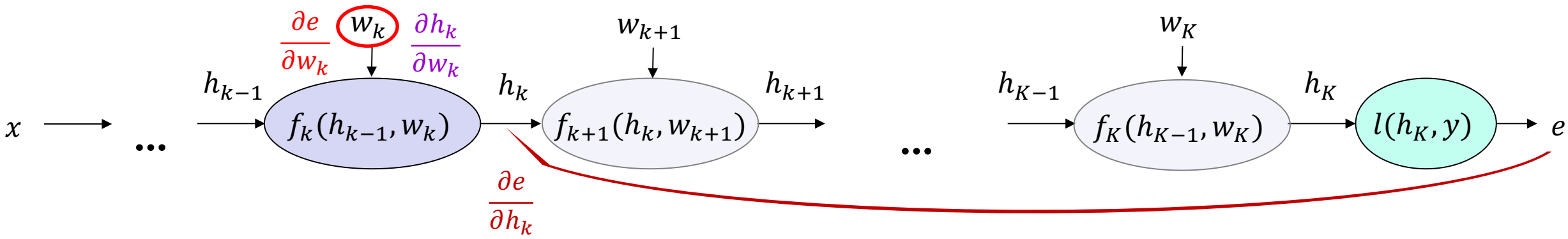
$e = l(h_1, y) = (y - h_1)^2$

$\frac{\partial h_1}{\partial w_1} =$

$\frac{\partial e}{\partial h_1} =$

$$\frac{\partial e}{\partial w_1} =$$

The Chain Rule for a Layer



$$\frac{\partial e}{\partial w_k} = \boxed{\frac{\partial e}{\partial h_K} \frac{\partial h_K}{\partial h_{K-1}} \cdots \frac{\partial h_{k+1}}{\partial h_k}} \frac{\partial h_k}{\partial w_k}$$

Upstream gradient $\frac{\partial e}{\partial h_k}$

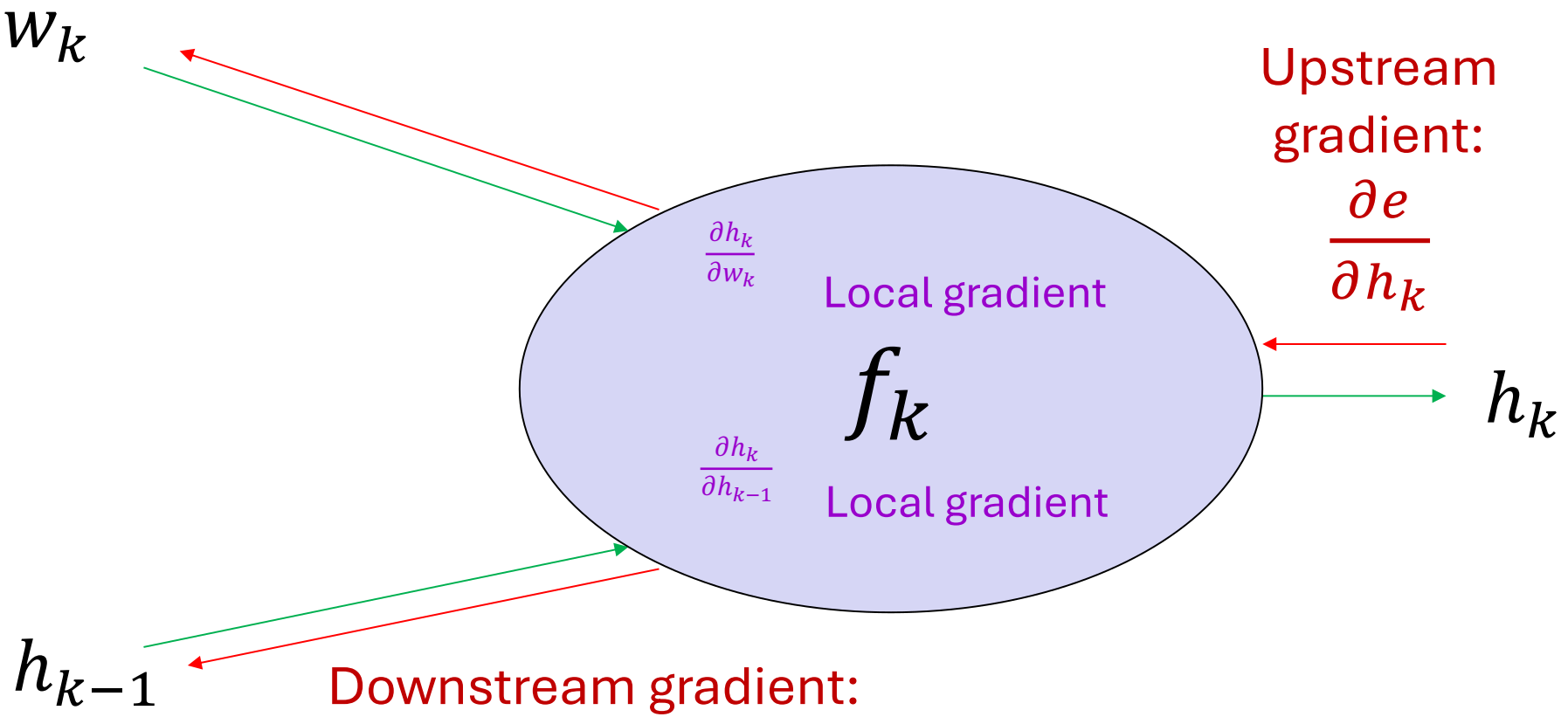
Local gradient

Adapted from S. Lazebnik, CS 444 (Spring 2025)

Backpropagation at One Layer

Parameter update:

$$\frac{\partial e}{\partial w_k} = \frac{\partial e}{\partial h_k}\frac{\partial h_k}{\partial w_k}$$



Downstream gradient:

$$\frac{\partial e}{\partial h_{k-1}} = \frac{\partial e}{\partial h_k}\frac{\partial h_k}{\partial h_{k-1}}$$

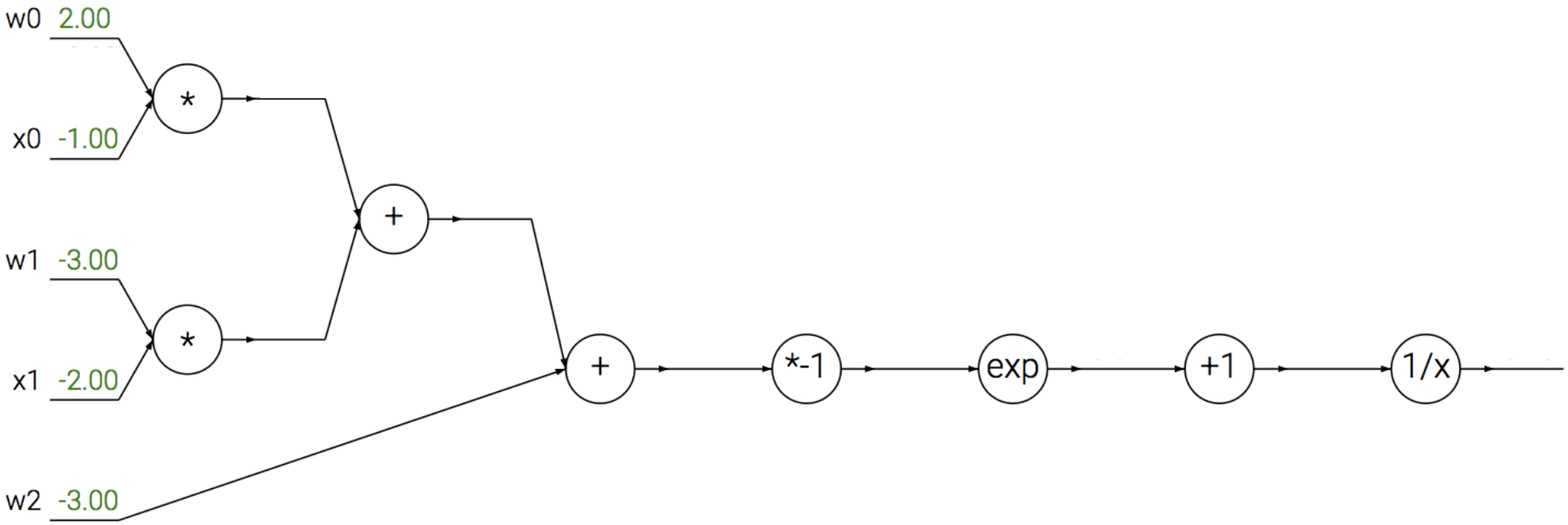
Forward pass

Backward pass

Adapted from S. Lazebnik, CS 444 (Spring 2025)

Worked Example: Backpropagation

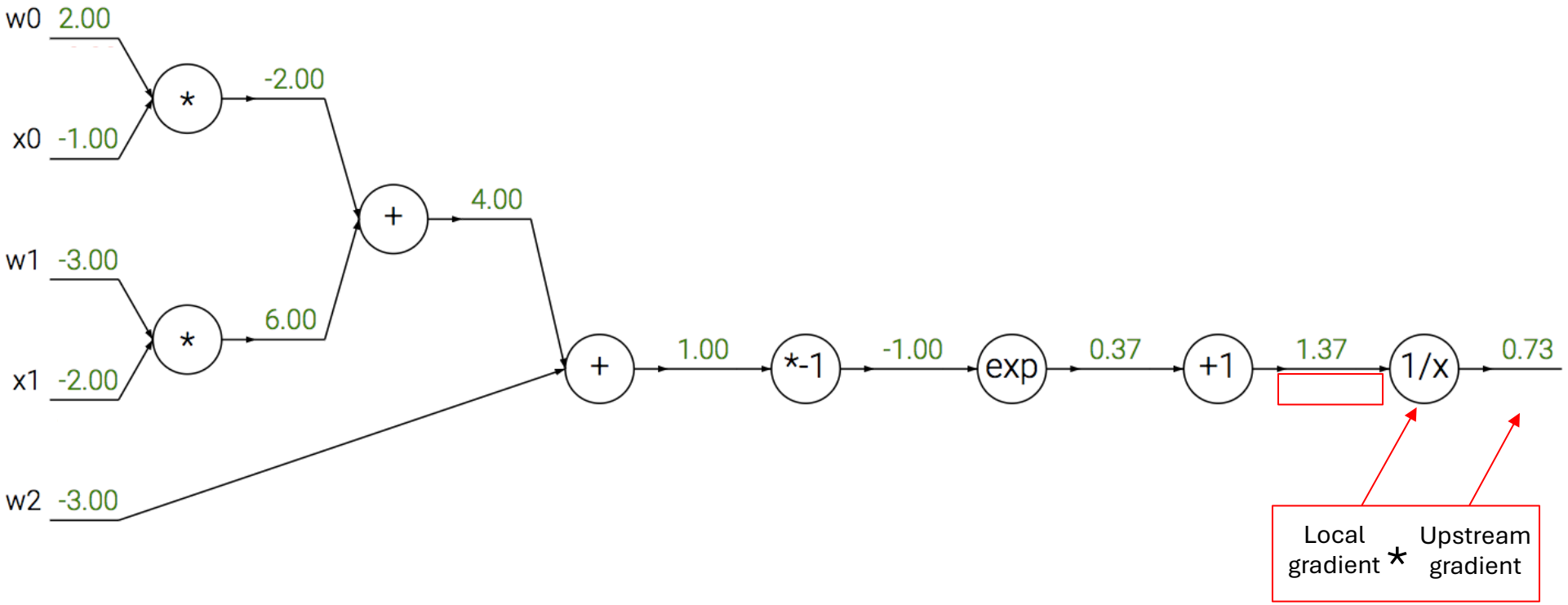
$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$



Source: Stanford 231n

Worked Example: Backpropagation

$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$



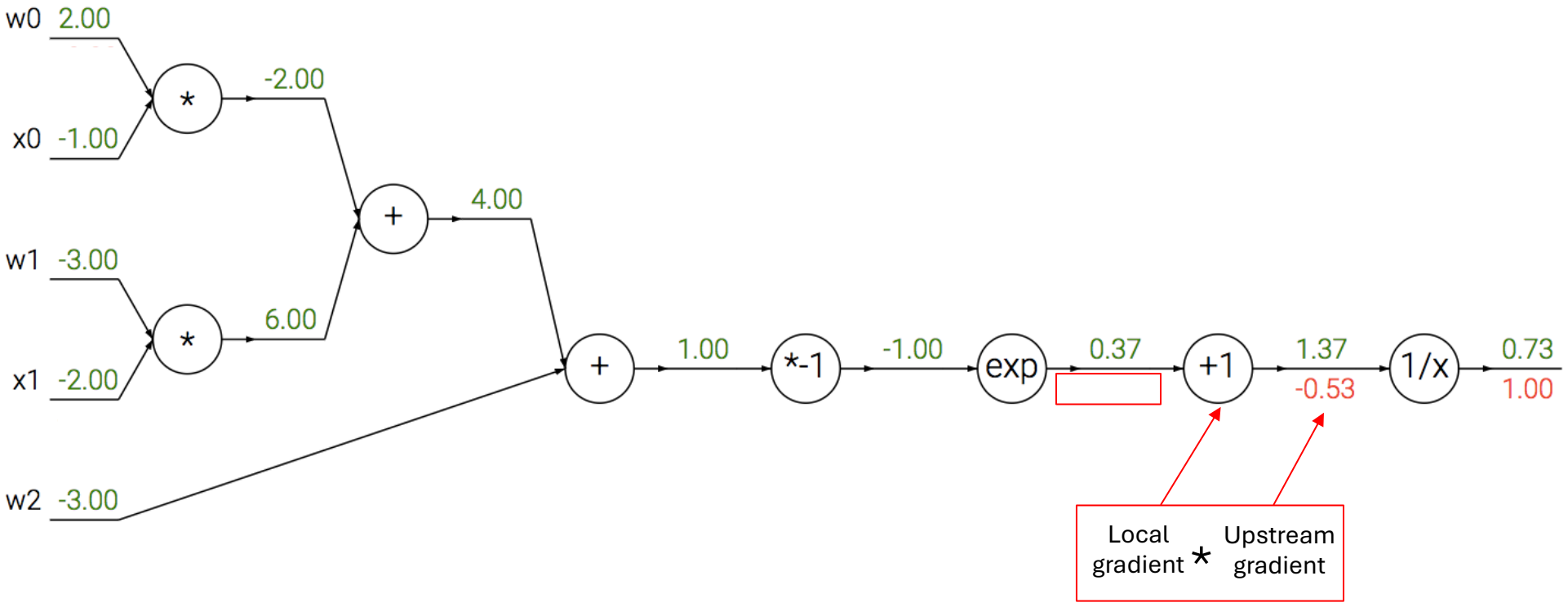
$$(1/x)' = -1/x^2$$

$$-\frac{1}{1.37^2} * 1 = -0.53$$

Source: Stanford 231n

Worked Example: Backpropagation

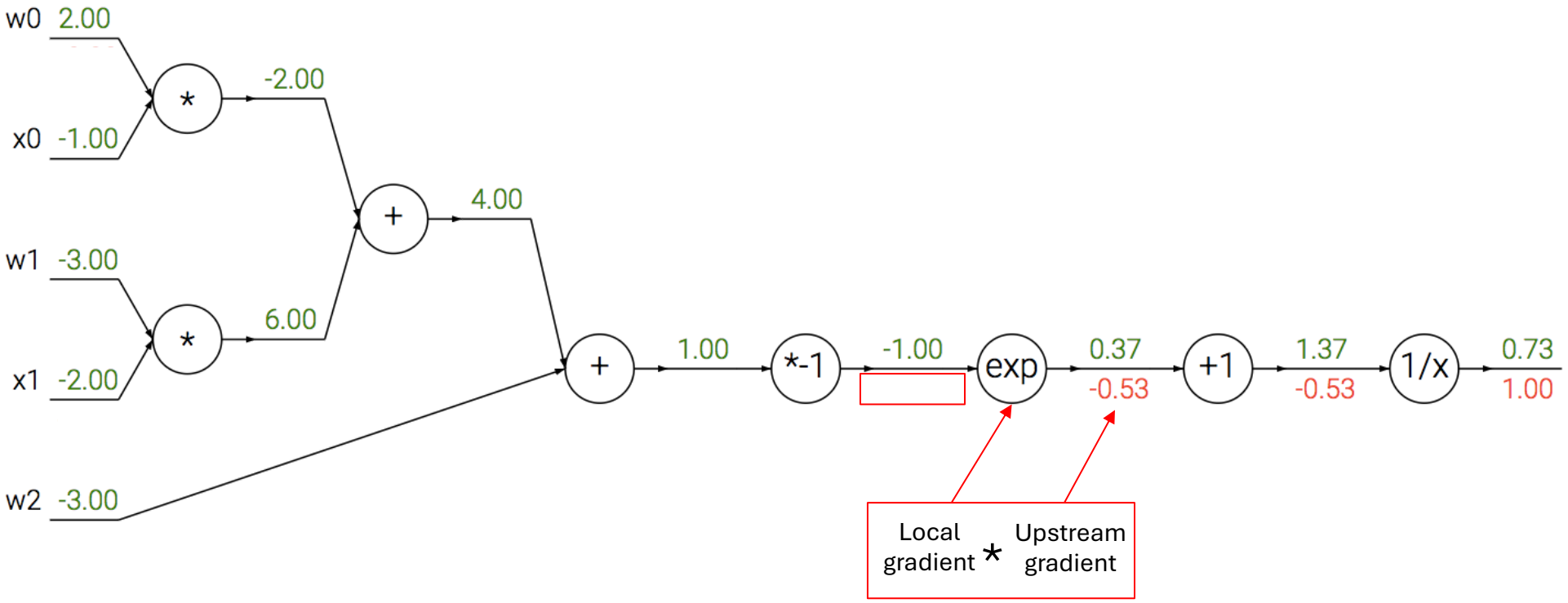
$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$



Source: Stanford 231n

Worked Example: Backpropagation

$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$



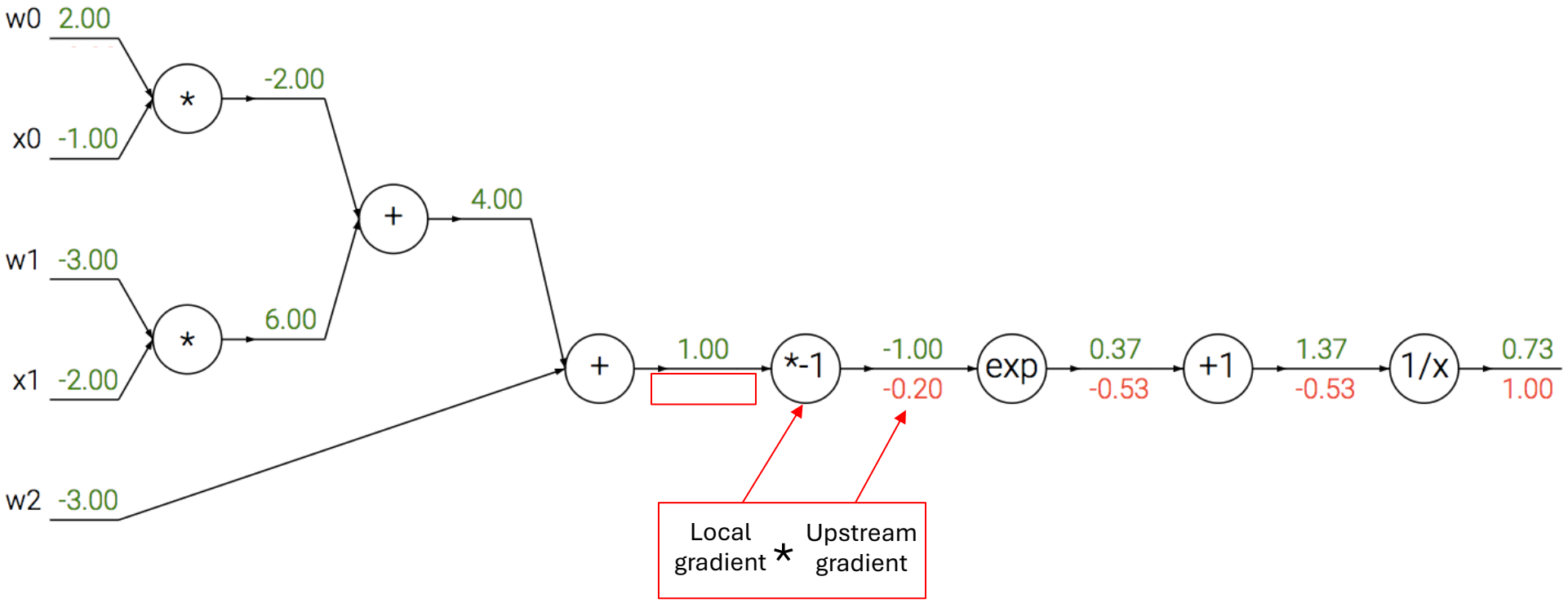
$$\exp(-1) * (-0.53) = -0.20$$

Adapted from S. Lazebnik, CS 444 (Spring 2025)

Source: Stanford 231n

Worked Example: Backpropagation

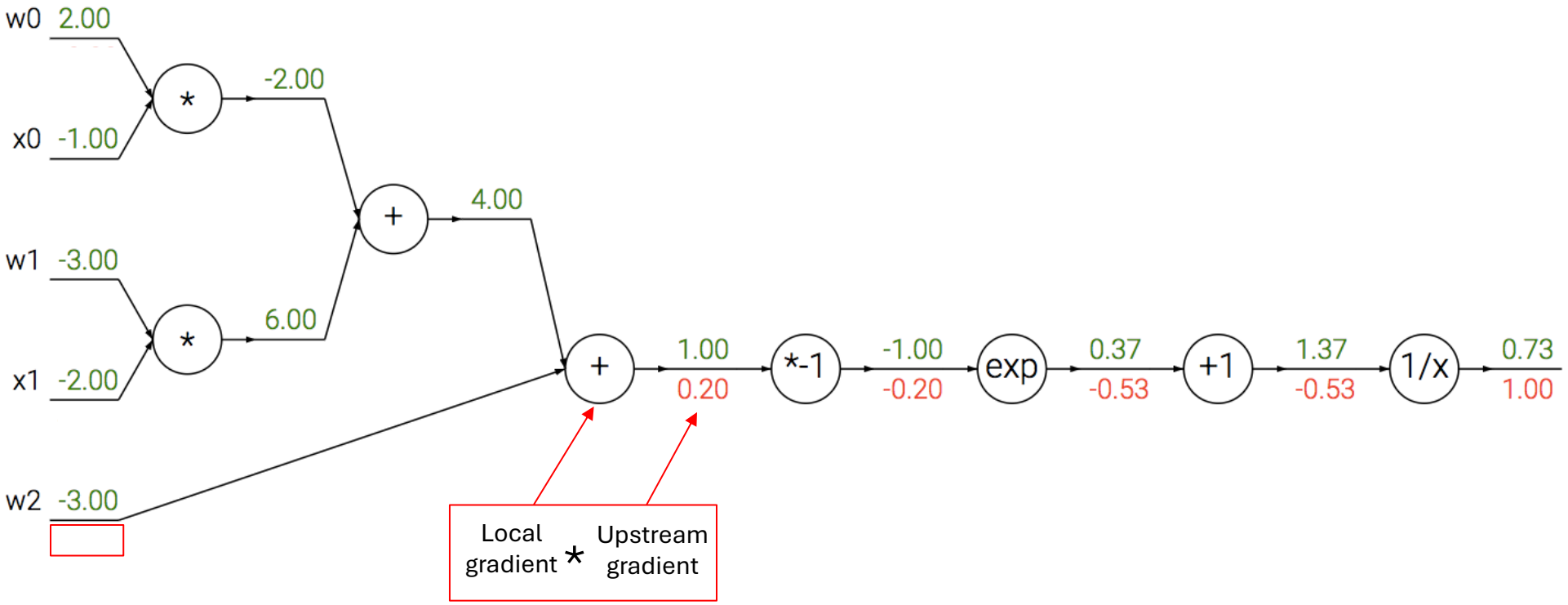
$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$



Source: Stanford 231n

Worked Example: Backpropagation

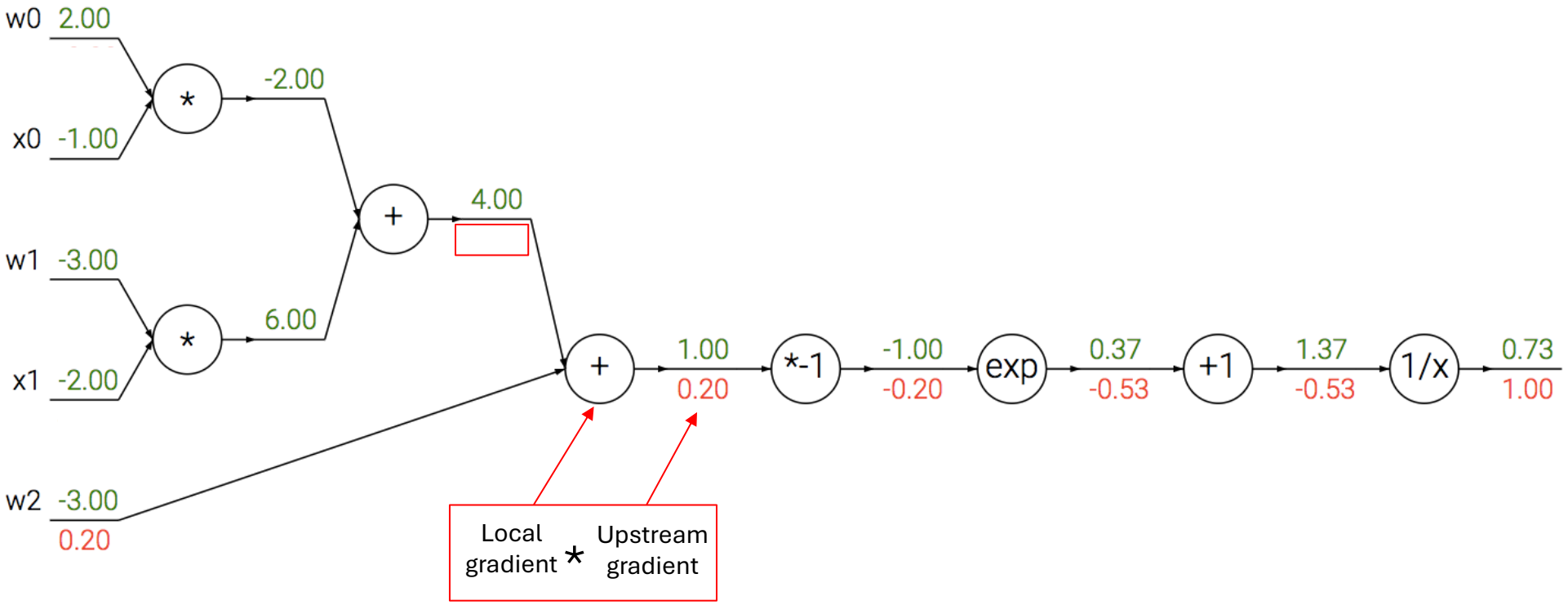
$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$



Source: Stanford 231n

Worked Example: Backpropagation

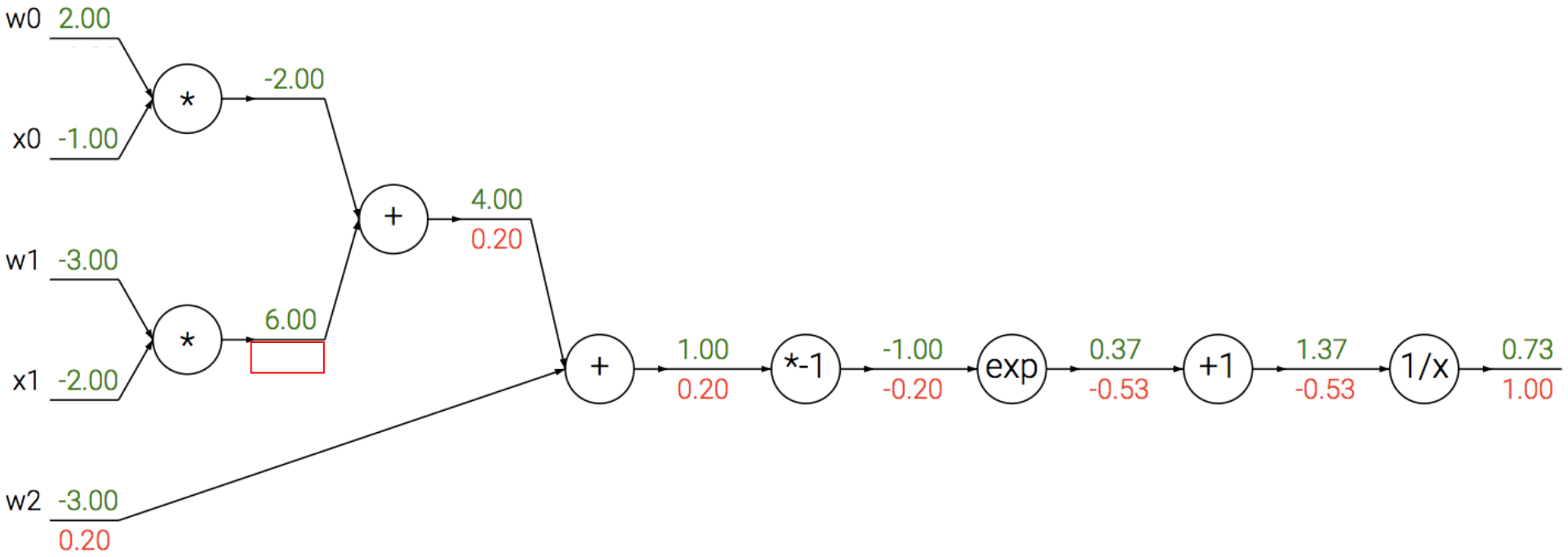
$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$



Source: Stanford 231n

Worked Example: Backpropagation

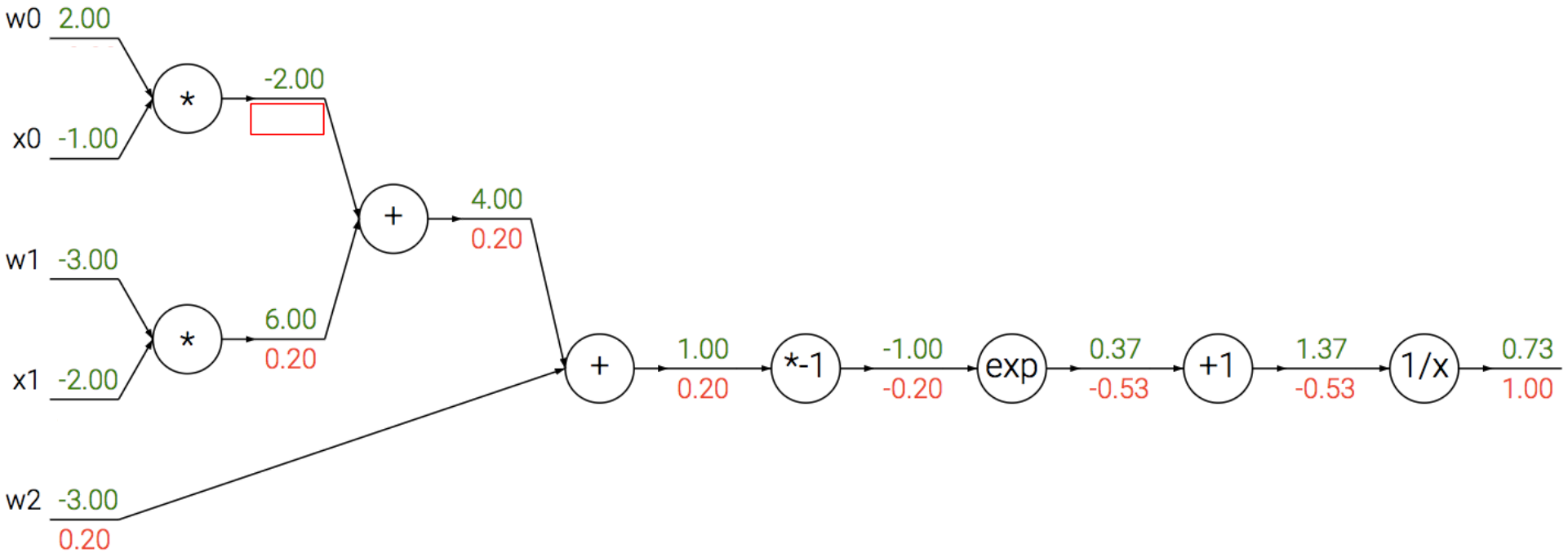
$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$



Source: Stanford 231n

Worked Example: Backpropagation

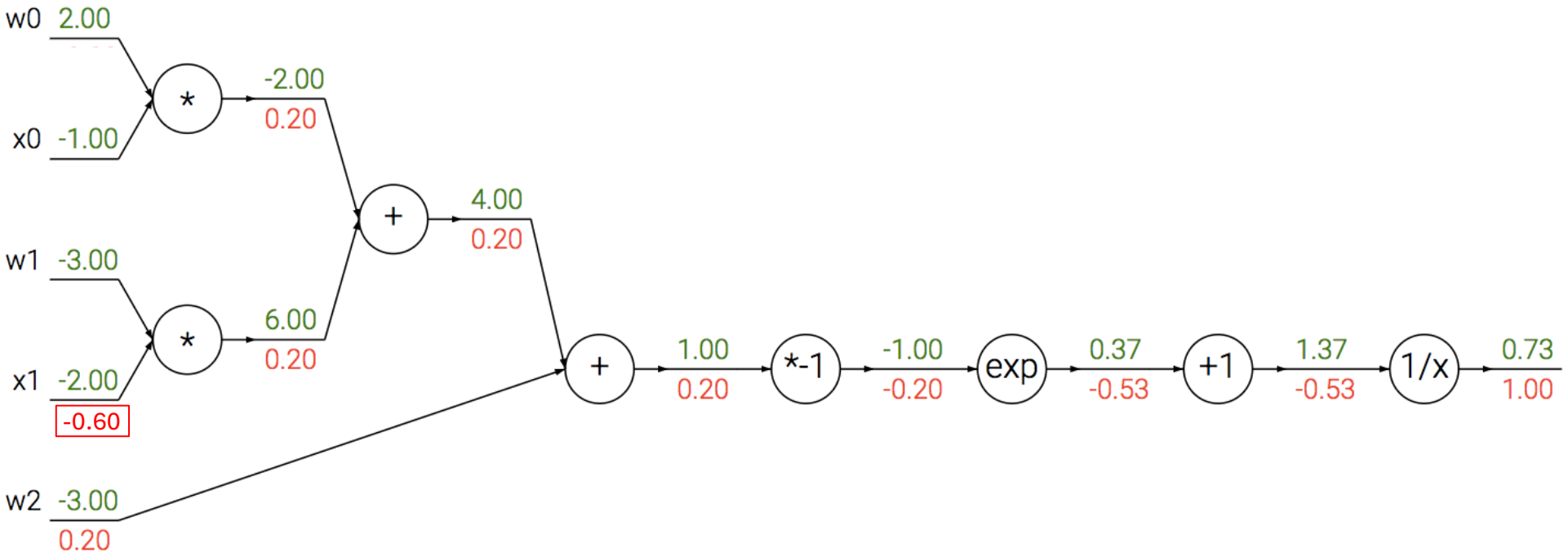
$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$



Source: Stanford 231n

Worked Example: Backpropagation

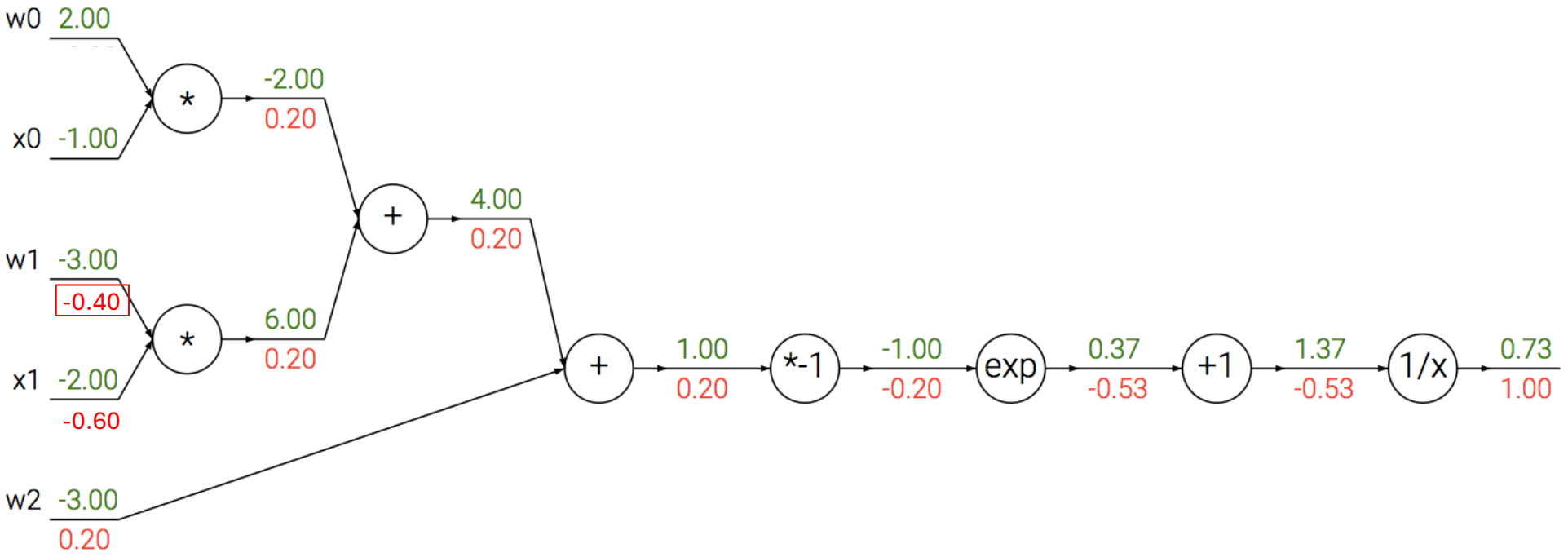
$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$



Source: Stanford 231n

Worked Example: Backpropagation

$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$

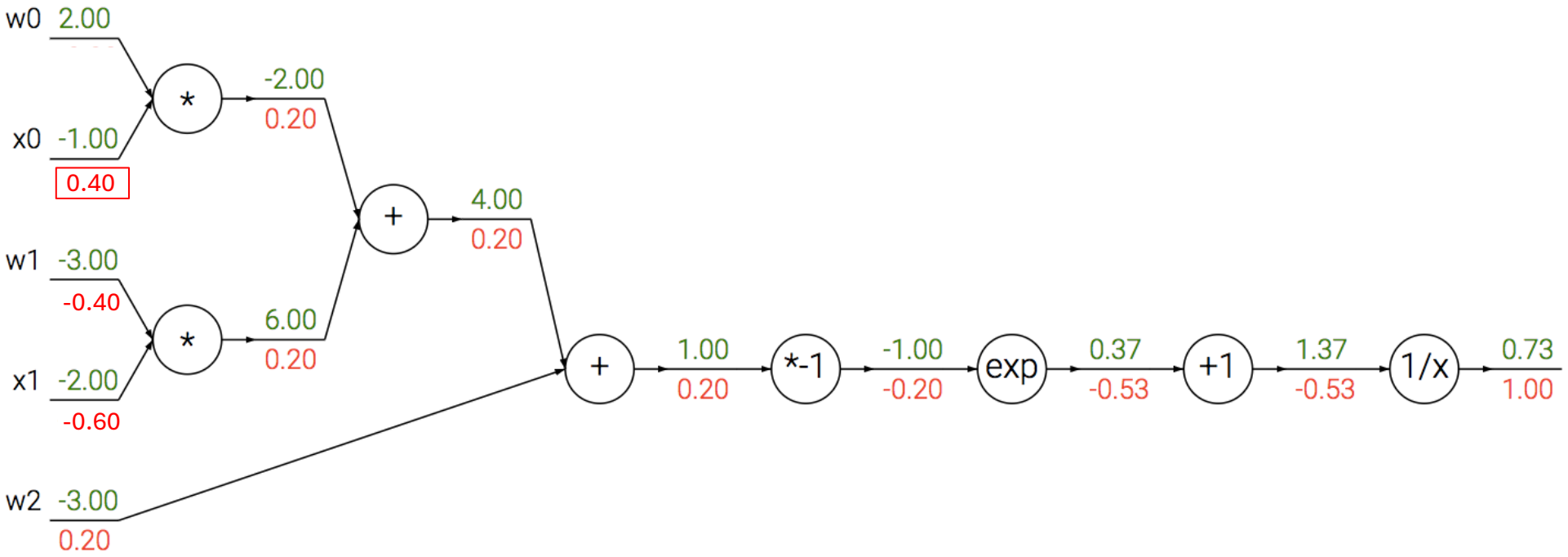


Adapted from S. Lazebnik, CS 444 (Spring 2025)

Source: Stanford 231n

Worked Example: Backpropagation

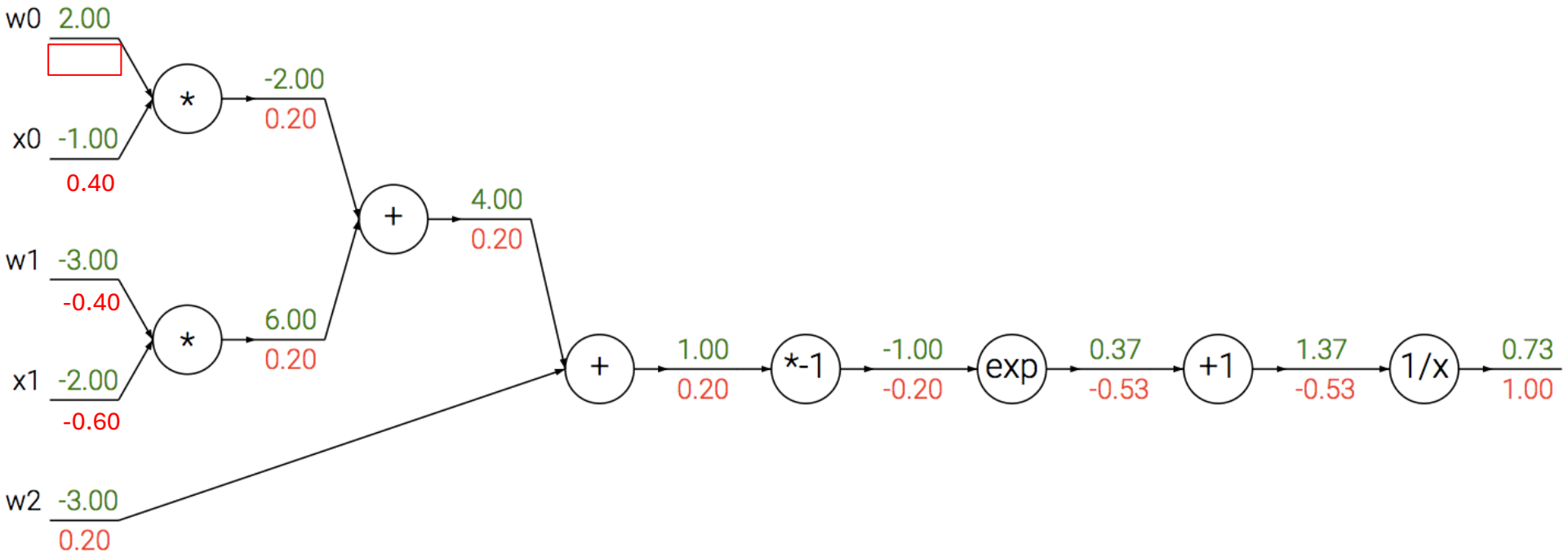
$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$



Source: Stanford 231n

Worked Example: Backpropagation

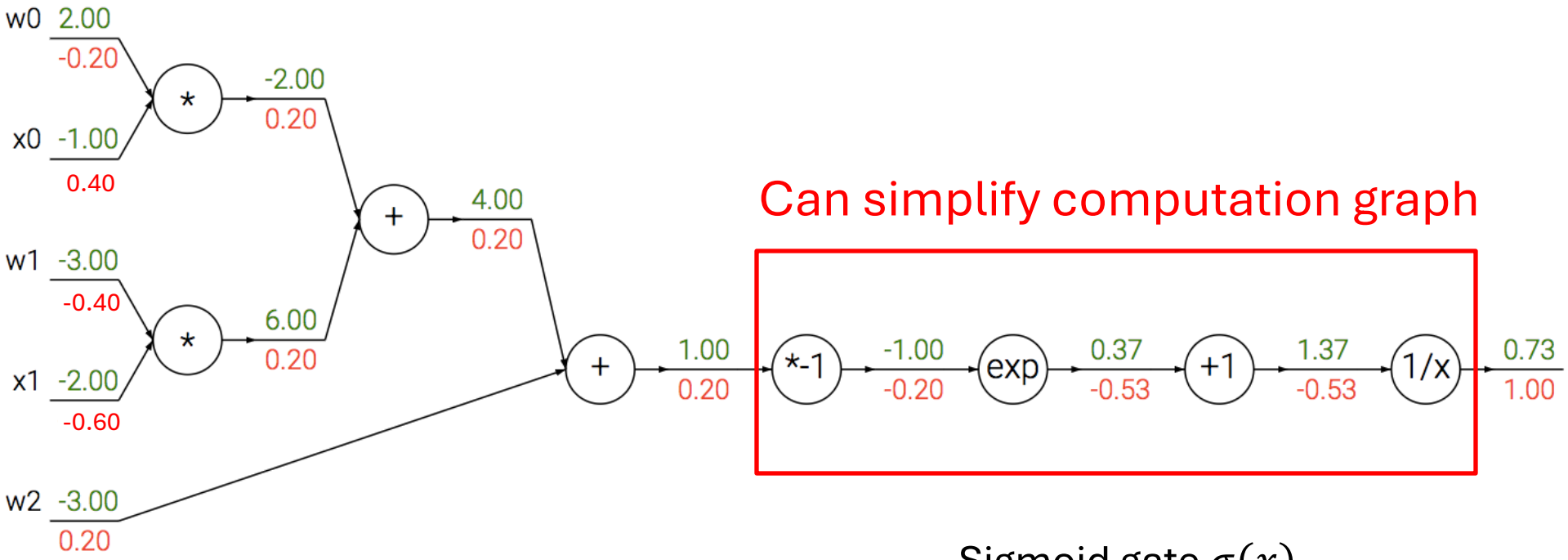
$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$



Source: Stanford 231n

Worked Example: Backpropagation

$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$



Sigmoid gate $\sigma(x)$

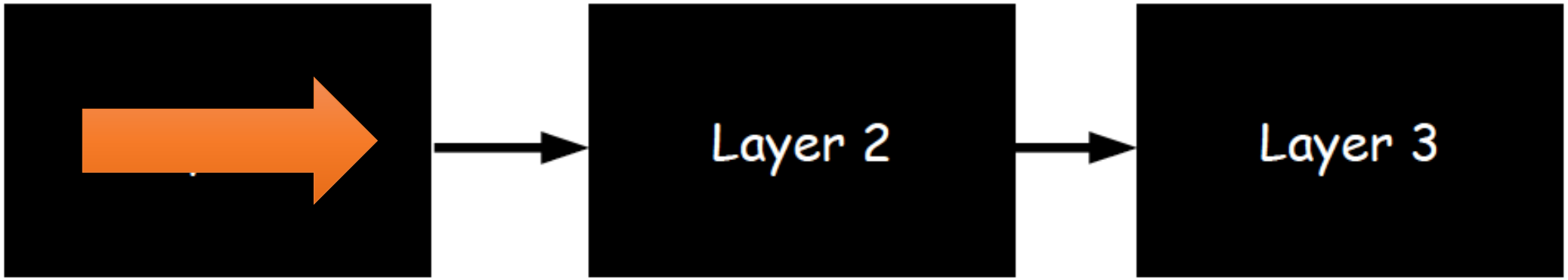
$$\sigma'(x) = \sigma(x)\big(1-\sigma(x)\big)$$

$$\sigma(1)\big(1-\sigma(1)\big) = 0.73 * (1-0.73) = 0.20$$

Source: Stanford 231n

Neural Network Training

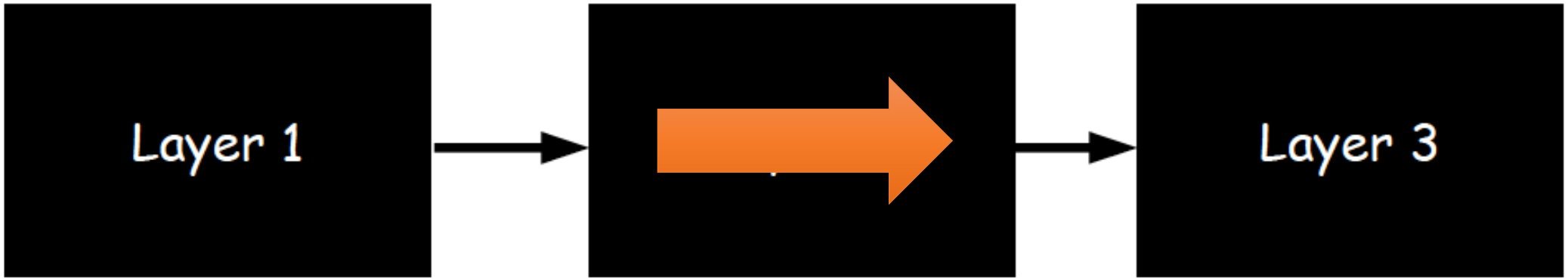
- Step 1: Compute Loss on mini-batch [F-Pass]



Slide Credit: Marc'Aurelio Ranzato, Yann LeCun

Neural Network Training

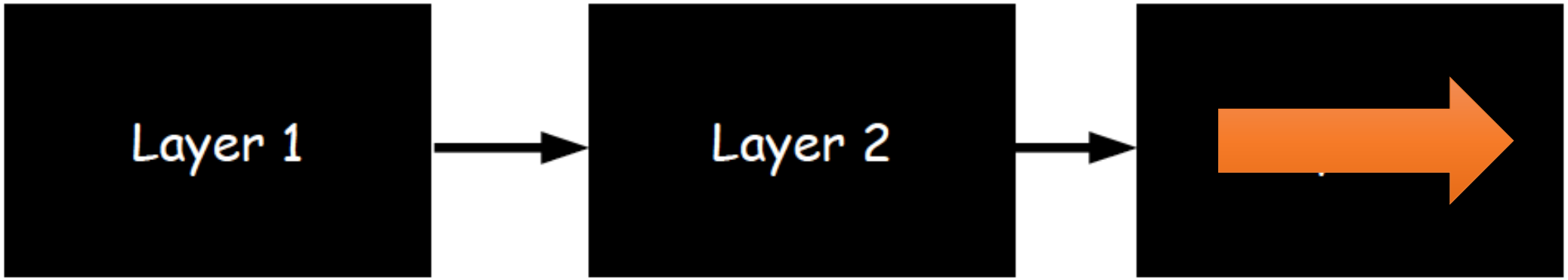
- Step 1: Compute Loss on mini-batch [F-Pass]



Slide Credit: Marc'Aurelio Ranzato, Yann LeCun

Neural Network Training

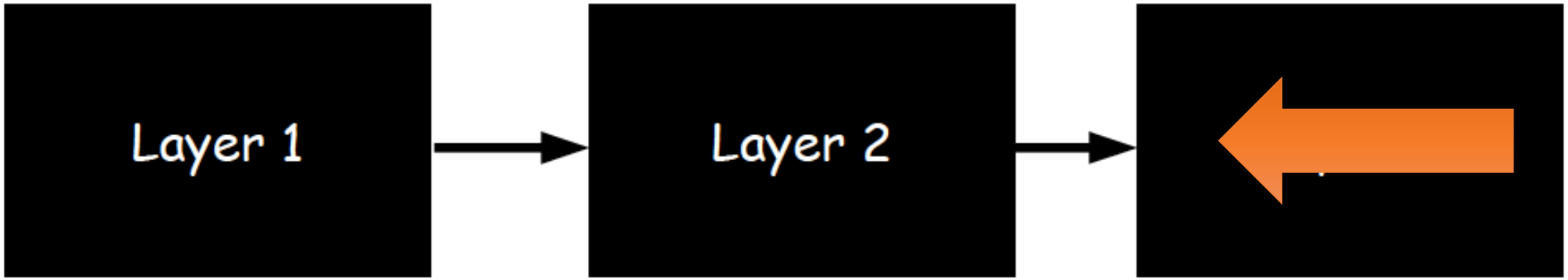
- Step 1: Compute Loss on mini-batch [F-Pass]



Slide Credit: Marc'Aurelio Ranzato, Yann LeCun

Neural Network Training

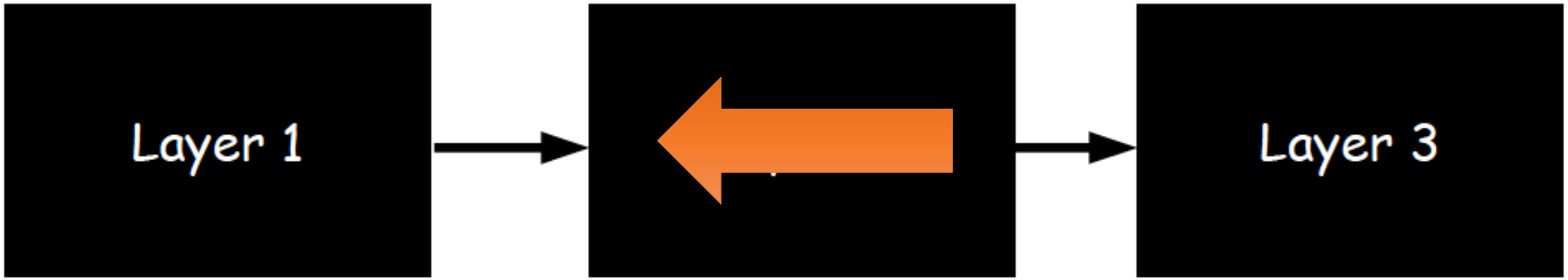
- Step 1: Compute Loss on mini-batch [F-Pass]
- Step 2: Compute gradients wrt parameters [B-Pass]



Slide Credit: Marc'Aurelio Ranzato, Yann LeCun

Neural Network Training

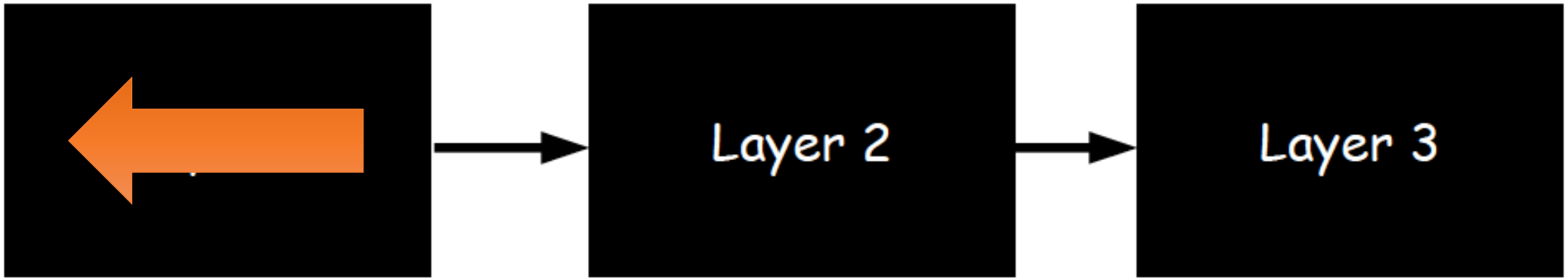
- Step 1: Compute Loss on mini-batch [F-Pass]
- Step 2: Compute gradients wrt parameters [B-Pass]



Slide Credit: Marc'Aurelio Ranzato, Yann LeCun

Neural Network Training

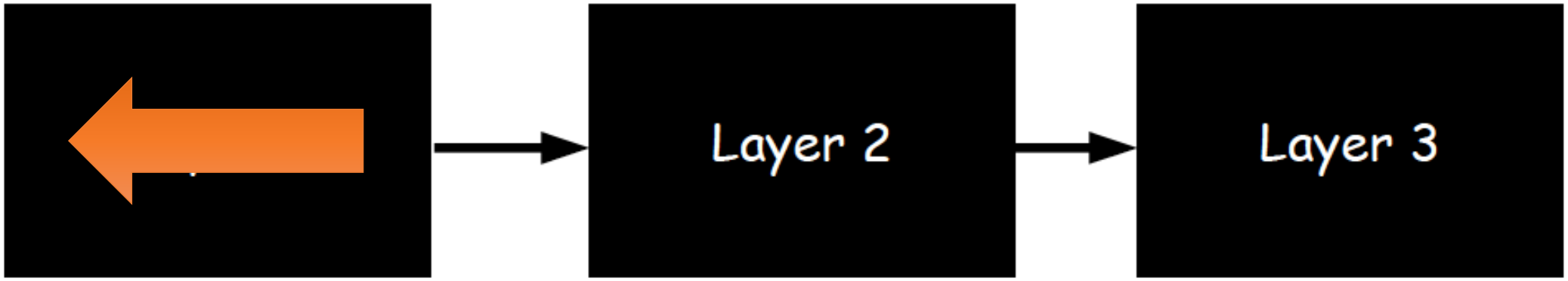
- Step 1: Compute Loss on mini-batch [F-Pass]
- Step 2: Compute gradients wrt parameters [B-Pass]



Slide Credit: Marc'Aurelio Ranzato, Yann LeCun

Neural Network Training

- Step 1: Compute Loss on mini-batch [F-Pass]
- Step 2: Compute gradients wrt parameters [B-Pass]
- Step 3: Use gradient to update parameters



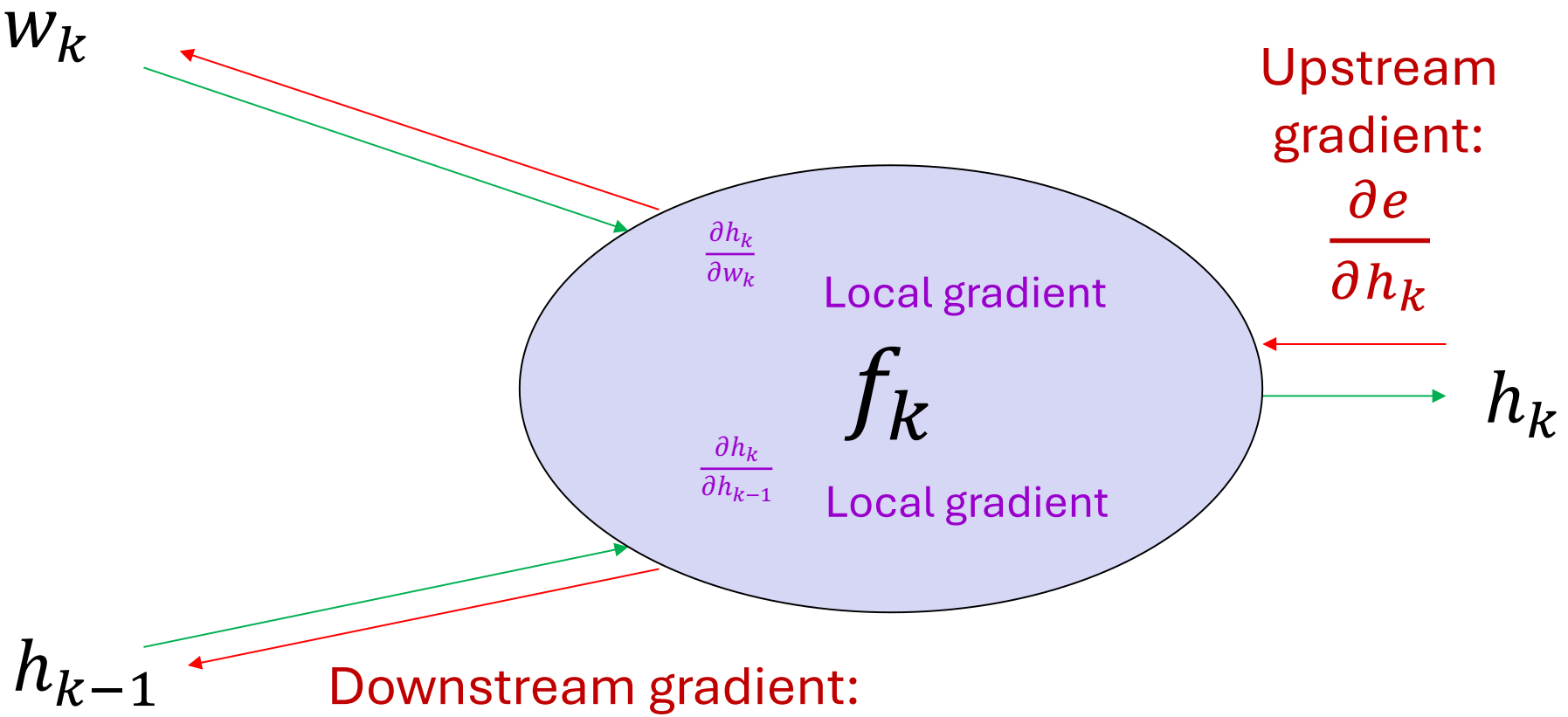
$$\theta \leftarrow \theta - \eta \frac{dL}{d\theta}$$

Slide Credit: Marc'Aurelio Ranzato, Yann LeCun

Backpropagation Summary

Parameter update:

$$\frac{\partial e}{\partial w_k} = \frac{\partial e}{\partial h_k}\frac{\partial h_k}{\partial w_k}$$



Downstream gradient:

$$\frac{\partial e}{\partial h_{k-1}} = \frac{\partial e}{\partial h_k}\frac{\partial h_k}{\partial h_{k-1}}$$

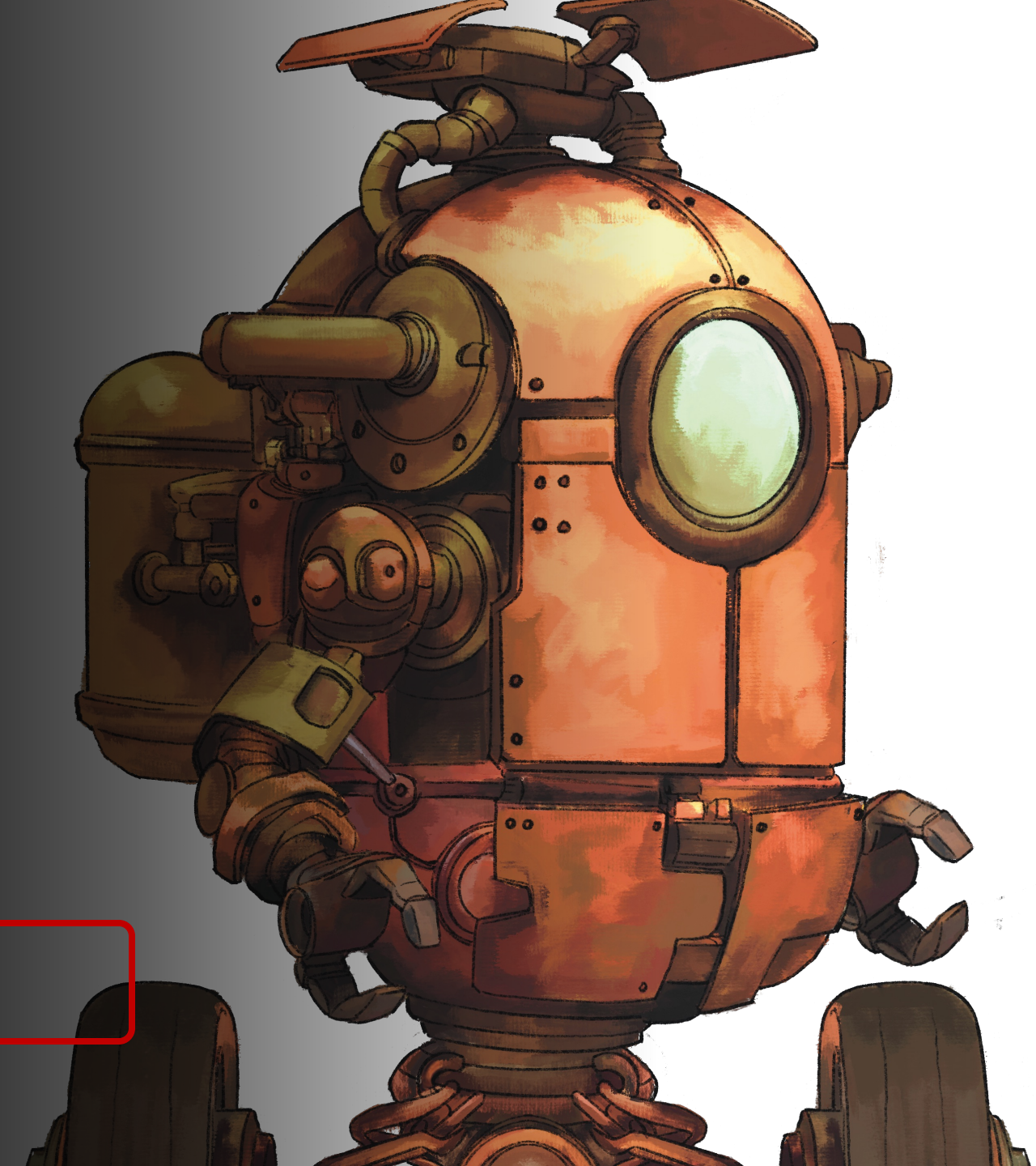
Forward pass

Backward pass

Adapted from S. Lazebnik, CS 444 (Spring 2025)

Outline

1. Intra-class variability
2. Supervised Learning
3. Models and Losses
4. Gradient Descent and Minibatches
5. Calculating Gradients
6. Transformers in Robotics



Transformers in Robotics

Attention lets each token draw on other tokens, with weights that depend on the input.

Language

Text tokens → next token

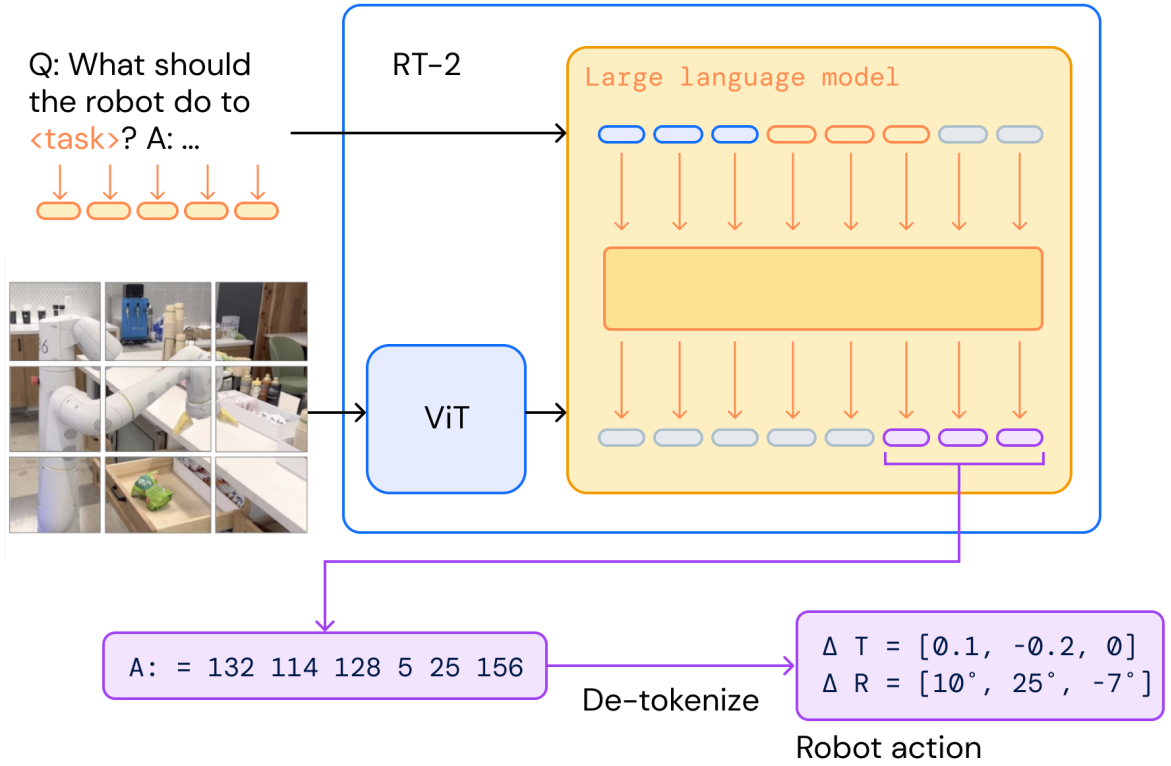
Vision

Image patches → class or visual features

Vision–language–action

Camera + instruction → robot action

A differential-drive policy can predict forward speed v and turn rate ω.



RT-2 inference • Brohan et al. (2023)

Can the model run fast enough on the robot's onboard computer?